

In questa materia sappiamo che oltre la teoria e il codice scritto su java, sono presenti i diagrammi UML. I diagrammi UML sono insiemi di rappresentazioni grafiche che servono per rappresentare un software. Java è un linguaggio abbastanza semplice che si basa sulla programmazione ad oggetti implementata su c++.

Hello World In Java

```
import java.time.LocalDate;

/**
 * Classe che stampa sullo schermo un messaggio e la data corrente
 */
public class HelloWorld { // definizione classe
    // dichiarazione e assegnazione campi
    private static final String msg = "Lezione di Ingegneria del Software";
    private static final LocalDate d = LocalDate.now();

    /**
     * Metodo da cui inizia l'esecuzione del programma
     *
     * @param args parametri passati al metodo all'avvio della classe
     */
    public static void main(String[] args) {
        System.out.println("Hello World");
        System.out.println(msg);
        System.out.println(d);
    }
}
```

```
Output
Hello World
Lezione di Ingegneria del Software
2025-03-03
```

10

Prof. Tramontana - Marzo 2025

Questo è un semplice programma che introduce una semplice stampa del famoso "Hello world" su java. Vediamo subito una cosa importante, che:

IL NOME DEL FILE DEVE ESSERE UGUALE AL NOME DELLA CLASSE, IN TUTTO E PER TUTTO, QUINDI ANCHE LE MINUSCOLE E MAIUSCOLE.

Varie definizioni:

- DESIGN PATTERN-> Ci da una soluzione di progettazione, collaudata e riusabile per un problema ricorrente di progettazione a oggetti.
- PROCESSO DI SVILUPPO-> EXTREME PROGRAMMI(EXP). Si intende le attività che vengono usate per la progettazione e manutenzione del software.

CAMBIAMENTI DEL CODICE->

Un codice quando viene ultimato, normalmente non si tocca più se funziona bene, ma in realtà ci sono vari motivi per la quale il codice può essere modificato:

- Rimozione degli errori(se presenti);
- Aggiunta di funzionalità in più nel codice;
- Aggiunta di sintassi o commenti nel codice per capirlo meglio;

In particolare nell'ultima tipo di modifica se viene aggiunta sintassi che non modifica il comportamento del codice, allora li si parla di **REFACTORING**.

Se invece i cambiamenti che vengono fatti al codice cambiano il comportamento del programma, magari per farlo adattare a nuove librerie, allora in quel caso si parla di **CAMBIAMENTI DI TIPO ADATTIVO**.

VARIE DEFINIZIONI E APPUNTI SU JAVA:

Ci sono vari comandi usati su java, e sono:

Parole Chiave Di Java

- **class** permette di definire un tipo, e quindi le sue istanze
- **final** definisce un campo o una variabile che non può essere assegnata più di una volta (una costante). Una classe final non può essere ereditata, un metodo final non può essere ridefinito (override)
- **import** indica dove trovare la definizione di una classe che sarà usata nel seguito
- **new** permette di creare un'istanza di una classe
- **private** e **public** indicano l'accessibilità di classi, campi e metodi
- **static** è usata per dichiarare un campo o un metodo appartenente alla classe (e non all'istanza)
- **void** indica che il metodo non ritorna alcun valore
- Tipi usati: **String**, per rappresentare insiemi di caratteri; **LocalDate**, per accedere alla data attuale; **System** per scrivere sullo schermo

```
1 public class Esempilezione {
2     private static int valore=0; //In questo caso sotto valore spunta un tratteggio giallo per evidenziare che questa variabile non è utilizzata
3
4     private int contatore; //Qui la variabile non è statica, quindi non si può mettere all'interno del metodo main
5
6     Run|Debug
7     public static void main(String[] args){ //Facciamo stampare un semplice Hello Word(sopra la funzione c'è il tasto di run e debug che direttamente fa runnare il programma cl
8         System.out.println("Hello Word"); //Su java quando si stampa si mette sempre System.out;
9         Esempilezione h=new Esempilezione();
10        h.stampe(); //Qui richiamiamo un'altra funzione esterna nel nostro main principale appartenente alla classe, sempre con una variabile di tipo classe
11    }
12
13    private void stampe(){
14        valore++; //Questo incrementa la variabile solo la prima esecuzione
15
16        System.out.println("Come state"); //La parola static(nel main) indica che quella funzione vive nella classe, quindi operiamo direttamente sulla classe
17
18        System.out.println("Valore:"+valore); //Qui nella sintassi c'è il più invece che la virgola;
19
20        /*Esempilezione esempio= new Esempilezione(); //Qui stiamo dichiarando una variabile locale al metodo main, ed è di tipo Esempilezione
21        esempio.contatore++; //Qui usiamo l'istanza che è nella classe(esempio) e mettiamo la variabile contatore che non è della classe, per farla diventare della classe
22        System.out.println("Contatore:"+esempio.contatore); //Qui stampiamo la variabile che abbiamo chiamato con l'istanza della classe
23
24        //Se chiamassimo esempio.valore quindi con la variabile di tipo classe, ad una variabile dentro la classe, non succede niente.
25
26        Esempilezione esempiol= new Esempilezione();
27
28        System.out.println(esempiol.contatore); //Qui stiamo chiamando con un'altra istanza della classe, alla stessa variabile contatore non statica
29    }
30
31    contatore++;
32    System.out.println("Contatore:"+contatore); //Qui lo stampiamo non usando la variabile di tipo classe che è già usata per chiamare la funzione stampe()
33
34    //La variabile contatore non è inizializzata, ma rispetto a c e c++, java inizializza le variabili automaticamente a 0.
35
36
37 }
```

Su java c'è una cosa molto importante e comoda, ossia che le variabili vengono automaticamente inizializzate a 0. Poi nel compilatore e debugger java vi sono dei comandi che sottolineano gli errori, oppure sottolineano con una linea gialla, una variabile che magari viene dichiarata ma non viene usata.

Guardando questo codice possiamo vedere come abbiamo una classe, con all'interno 2 funzioni, la funzione main e la funzione stampa. Quando chiamiamo il main con static, significa che quel metodo appartiene a quella classe a prescindere da tutto, quindi qualsiasi comando inserito lì dentro viene subito eseguito. Nel caso della funzione stampa chiamata solo con private, in quel caso dovrà essere chiamata funzione all'interno del main con un oggetto di tipo classe:

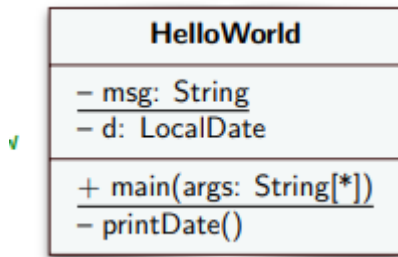
ES nell'immagine->h.stampe();

Quando dichiariamo le variabili con tipi primitivi(int,double,float,char) questi tipi vanno scritti con la minuscola iniziale, quindi non Double ma double.

Per String invece, deve essere scritto sempre con la maiuscola iniziale.

ALTRE DEFINIZIONI:

Diagramma UML-> Una classe nei diagrammi UML viene descritta come un rettangolo, in cui all'interno andiamo a mettere le varie variabili, funzioni ecc...



il + nel nostro rettangolo significa pubblico, invece il - è privato.

-La sintassi è: visibilità, nome metodo e gli attributi.

-Gli attributi o metodi quando sono statici vengono sottolineati.

-Tra un parametro e l'altro dentro un metodo viene messo un punto e virgola(;)

CARATTERISTICHE DEL SOFTWARE:

Come già enunciato prima il software è modificabile e può subire variazioni di vario tipo, grazie al fatto che non possiede parti fisiche, e si può adattare a qualsiasi tipo di hardware. Questi cambiamenti sono dati dal successo che un software ha. Se un software ha tanto successo ha ovviamente bisogno di cambiamenti(aggiornamenti) proprio come in un videogioco di successo.

QUALITÀ DEL SOFTWARE->

Le qualità del software vengono indicate da diverse caratteristiche, che sono:

CORRETTEZZA-> La correttezza del software indica se un software venduto ad un cliente svolge realmente quello che dovrebbe fare, e quindi è corretto;

EFFICIENZA->Nessuno spreco inutile di risorse;

MANUTENIBILITÀ->Facilità ad introdurre modifiche;

SICUREZZA->Che si divide in safety: Che è la sicurezza delle persone dalla parte hardware su cui è installato il software; Security: È la capacità di difendere i dati del sistema(Limita gli accessi);

USABILITÀ->Indica quanto è facile il software da usare.

La capacità del software di essere modificato, è già una qualità che un software deve avere a prescindere.

INTERFACCIA-> L'interfaccia in java definisce un tipo senza implementarne il codice. Ad esempio l'interfaccia LIST definisce vari metodi tra cui add(), get(), size(), ossia tutti i metodi utili per una lista. La classe ArrayList è una classe che implementa l'interfaccia list.

Un buon principio di programmazione suggerisce che è meglio programmare per le interfacce e non per le implementazioni, cioè è più utile inserire più interfacce che magari

implementano con se i vari metodi, che scrivere molti metodi uno diverso dall'altro, che poi sfavoriscono un cambiamento adattivo del programma, ossia che se devo cambiare il programma dovrò andare a cambiare il doppio delle cose rispetto all'interfaccia. Questo permette anche di implementare più facilmente i test.

In UML sappiamo che quando vediamo un rettangolo, quello rappresenta una classe, ma in realtà se al suo interno viene scritto << interface, allora quella lì è un'interfaccia. Il vero e proprio simbolo dell'interfaccia in UML è il cerchio, infatti possiamo vederla rappresentata o con un cerchio con dentro LIST, oppure con il rettangolo con al suo interno la parola LIST e un cerchio.

Un'altra cosa molto importante ed utile di java è che anche se nel corso del nostro piano di studi abbiamo imparato ad allocare strutture dati come pile, liste, code ecc.. , adesso non servirà più, dato che java tramite l'importazione di librerie alloca automaticamente lo spazio per ogni nuovo oggetto in una struttura dati. Basta mettere la sintassi List < tipo>

La lista verrà sempre dichiarata nella classe principale con la sintassi:

```
private final List < String >nomeLista = new ArrayList <> ();
```

Successivamente aggiorniamo la lista aggiungendo degli elementi, o manualmente oppure leggendo un file, ma questo viene fatto in un metodo a parte.

Nella lista vi è anche un metodo chiamato contains() che verifica se quell'elemento inserito in input è presente nella lista.

readLine() è un metodo della libreria LineNumberReader che ritorna la stringa contenuta in quella specifica riga del file.

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.io.BufferedReader;
import java.util.ArrayList;
import java.util.List;
```

Queste sono alcune delle librerie che vengono implementate in un programma in java, tra cui sono presenti quelle per leggere i file ecc... . Quelle indispensabili per implementare le liste sono:

-java.util.ArrayList;

-java.util.List

Ci sono 2 nuovi termini nuovi in java, che sono TRY E CATCH che sono collegati tra loro, dato che si usa quando la chiamata di metodi potrebbe lanciare un'eccezione, e quest'ultima viene catturata da catch.

SPAGHETTI CODE VS OOP:

Nel pdf vediamo come si esce la differenza tra una programmazione implementata con lo SPAGHETTI CODE, e un'implementazione fatta con la OOP fatta come si deve. Lo spaghetti code è un antipattern, ed è un codice monolitico, ossia che realizza tante cose nello stesso flusso, quindi non con la tecnica del OOP, e quindi non si può ne riusare ne verificarne la

correttezza. Oltre che i controlli verrebbero implementati in una maniera complicata, sarebbe difficile anche comprendere il codice stesso. Il problema più grande sta nel fatto che lo spaghetti code si implementa con un unico metodo, ed è per questo che è confusionario e complicato, e ci sono poche interazioni tra i vari oggetti, dato che nello spaghetti code si utilizzano di più le variabili globali.

Con la OOP invece si fanno molti meno metodi che poi sono riusabili, non si utilizzano variabili globali, sono facilmente comprensibili, e semplici da correggere. Con variabili di tipo della classe posso chiamare ogni singolo metodo per svolgere quello che voglio fare, ma soprattutto il vantaggio sta nel fatto che i metodi sono molto corti.

Qui si usa il paradigma command and query, ossia che:

- I metodi query chiedono delle cose, ma danno indietro il risultato, sono leggeri da eseguire e non modificano lo stato del sistema;
- I metodi command invece chiedono qualcosa ma non restituiscono nessun risultato, e cambiano lo stato del sistema. Il command forma il lavoro(cicli, ecc..) e si chiama solo quando serve.

L'estensione di funzionalità mediante sottoclassi viene chiamato riuso a white-box, ma richiede una perfetta comprensione della superclasse, per questo certe volte conviene utilizzare la composizione invece che l'ereditarietà, per renderlo più semplice.

TEST-> Quando si parla della correttezza di un programma non si parla solamente dell'utilizzo di molti metodi per rendere il codice più leggibile o riusabile o la separazione di command e query, ma si intende anche l'utilizzo dei TEST che sono fondamentali in java.

TEST-> È una verifica del funzionamento del sistema software, che verifica se il comportamento del codice è corretto o meno(il test viene fatto su tutto il programma e non su un singolo metodo). Esso esegue il codice sotto certe condizioni(variabili in input). Ogni test viene scritto in un ambiente controllato altrimenti i test non sono ben scritti e perdono di validità. Ovviamente il test deve dirci se è andato a buon fine o no perché sennò non sapremmo mai quando viene eseguito.

```

public class TestPagamenti { // per classe Pagamenti vers 0.9
    private Pagamenti pgm = new Pagamenti();

    private void initLista() {
        pgm.svuota();
        pgm.inserisci("321.01");
        pgm.inserisci("531.7");
        pgm.inserisci("1234.5");
    }

    public void testSommaValori() {
        initLista();
        pgm.calcolaSomma();
        if (pgm.getSomma() == 2087.21)
            System.out.println("OK test somma val");
        else System.out.println("FAILED test somma val");
    }

    public void testMaxValore() {
        initLista();
        pgm.calcolaMassimo();
        if (Math.abs(pgm.getMassimo() - 1234.5) < 0.01)
            System.out.println("OK test massimo val");
        else System.out.println("FAILED test massimo val");
    }
}
// continua ...

```

2

Prof. Tramontana - Marzo 2025

Un test quindi NON È una specifica di java implementata con una libreria, ma semplicemente un vero e proprio test implementato con dei metodi che permettono di verificare al meglio la correttezza del codice. Il test viene verificato tramite l'oracolo, che nei casi di test banali l'oracolo è il programmatore stesso che verifica che il test sia andato a buon fine, altrimenti si mettono dei controlli che verifichino che quel test sia andato a buon fine. Per fare questi test vi si deve fare proprio una classe a parte dove si implementano tutti i test. I test devono essere indipendenti tra loro e devono autovalutarsi. Una cosa importante in cui aiutano molto il programmatore, è che i test sono ottimi per evitare modifiche indesiderate, ciò significa che se io modificassi per sbaglio un valore tramite il println del test capirei dove si trova la modifica indesiderata. Si deve scrivere ed eseguire un test alla volta, così che se ci sono problemi si passa alla modifica del programma per togliere i problemi. Nell'esempio sopra abbiamo i test implementati nello stesso file del software che abbiamo scritto, e poi vengono chiamati dal main per essere eseguiti, ma in realtà noi abbiamo J-UNIT su java che li rende eseguibili automaticamente, quindi senza chiamarli nel main. Esistono due tipi di test:

- Test white-box: chi scrive i test conosce gli algoritmi che sta testando, immaginando come si comporterà. In genere li scrive il programmatore che sta sviluppando il componente.

- Test black-box: chi scrive i test non conosce l'algoritmo, ha solo il documento dei requisiti potendo immaginare tante modalità di esecuzione e i casi particolari. Li scrive un programmatore diverso da chi ha scritto l'algoritmo, potendo quindi essere ancor più oggettivo e ragionare fuori dagli schemi di chi ha scritto il programma. Saper scrivere test black-box significa essere esperti sulla scrittura del codice, perché immaginerà quali sono i punti critici del codice.

Il TDD(test-driven-development) è una tecnica che prima implementa i test(leggendo i requisiti che il programma vuole) e successivamente si implementa il codice dando i nomi

alle classi e ai metodi.

Il motto è: **RED**, **GREEN**, REFACTOR->

RED-> Il test non è superato, quindi si può andare a ritoccare il codice per far andare a buon fine il test;

GREEN-> Test superato, quindi o si fanno altri test o si passa al refactoring;

REFACTORING-> Si migliora la struttura del codice.

RESPONSABILITÀ-> Abbiamo detto precedentemente che un metodo può contenere solamente un compito, e un programma deve essere suddiviso su più classi per garantire la pulizia, comprensione e l'ereditarietà del codice che stiamo guardando. Un modo per capire se una classe ha una singola responsabilità è, scrivere una frase schietta dove si descrive quello che la classe compie, quindi non devono esserci congiunzioni, ma solamente un singolo ruolo.

PRINCIPI INGEGNERIA SOFTWARE-> Ci sono vari principi per garantire la buona programmazione di un software, e questi principi sono:

-KISS(keep it simple, stupid)-> Ciò significa che molti programmatori tendono a fare over-engineering, ossia tendono a complicare molto di più il codice rispetto a quanto lo sia veramente. Questo si deve evitare, perché oltre che sarebbe più comprensibile per chi legge, ma anche perché si evita l'avvenimento di bug.

-DRY(don't repeat yourself)-> bisogna sempre evitare le ripetizioni nel codice, infatti la stessa parola significa "asciutto"(con poco contenuto);

-YAGNI(you ain't gonna give it)-> Non bisogna implementare funzionalità che in quel momento non ti sono utili, ci si deve basare sui requisiti che il codice chiede, ma soprattutto dal momento.

-LoD(Law of Demeter)-> Ossia non parlare con gli sconosciuti ma solo con oggetti che sono amici, significa che ogni oggetto deve essere correlato ad un suo amico, e non ad un estraneo.

Se una classe parla con più classi allora va bene, ma se più classi parlano tutte con una singola classe, allora in quel caso non è ottimale.

PRINCIPI PER I TEST->

FIRST-> Sono i più importanti, e sono fondamentali perché ogni test deve necessariamente averli per essere considerato un ottimo test->

fast: deve ovviamente avere velocità, se un test ci impiega molto tempo a svolgersi, allora non è un buon test.

Isolato/indipendente: Ogni test deve dipendere solo da se stesso, cioè il suo risultato non deve dipendere da un altro test, e quindi se fallisce un test non significa che lui deve fallire per conseguenza.

Ripetibile: Un test deve essere sempre ripetibile, e deve dare lo stesso identico risultato su qualunque software stia girando.

Self-validating: Alla fine del test, il computer deve semplicemente dirti se il test è andato a buon fine o no, non deve essere il programmatore a doverlo leggere.

Timely-> I test devono essere scritti sempre al momento giusto.

TDD(test-driven-development)-> Quello che abbiamo introdotto prima con red, green e refactoring, e in base all'esito si svolge la conseguenza.

BDD->Significa "Sviluppo Guidato dal Comportamento". È una variante del TDD, ma invece di concentrarsi sui tecnicismi del codice, si concentra su **cosa deve fare il sistema dal punto di vista dell'utente**. **Un esempio pratico:** Immagina di testare un bancomat.

- **Given (Dato un contesto):** *Dato che* il mio conto ha 100€ e il bancomat ha contanti sufficienti...
- **When (Quando succede un evento):** *Quando* richiedo di prelevare 20€...
- **Then (Allora la risposta del sistema):** *Allora* il bancomat deve darmi 20€ e il mio nuovo saldo deve essere di 80€.

J-UNIT-> È un modo nuovo per facilitare la scrittura dei test. È un framework per i test di applicazione java, usata per eseguire test su classi e metodi. Quindi il TDD è la teoria, e J-UNIT è la pratica. Il paradigma che segue J-UNIT è proprio quello di implementare una classe per i test su cui provarli.

Le chiavi principali su come usare J-UNIT sono:

ANNOTAZIONI-> Sono una libreria a parte da installare su java, perché non sono uno standard su java. Tramite esso il programmatore dialoga con il compilatore. Tramite l'etichetta @Test messo sopra un metodo, fa capire al compilatore che quello lì non è un semplice metodo ma un test che deve essere eseguito.

ASSERZIONI-> Sono il vero cuore del test, e sono comandi come (AssertEquals, AssertTrue, AssertFalse, AssertNull), e verifica che il test vada a buon fine. È una classe implementata su j-unit, quindi quando implementiamo la libreria si implementa la classe.

TEST-RUNNER->È il motore di JUnit. Con un solo clic, prende tutti i centinaia (o migliaia) di test che hai scritto e li esegue alla velocità della luce (rispettando il principio **Fast** della regola FIRST). Icona verde, rossa.

Per interagire con j-unit devo creare delle cartelle e ne creo una chiamata SRC dove avremo una con cartella TEST e una con cartella MAIN. Nella cartella main mettiamo tutte le classi dell'applicazione, e in quella TEST mettiamo i test. Verrà anche messa una tabella target dove verranno messi gli eseguibili e verrà creata automaticamente alla prima esecuzione.

Per verificare il comportamento del codice si utilizzano le asserzioni, e quindi i metodi della classe assertion, come:

assertEquals(expected, actual) -> questa è l'asserzione più importante, dato che indica il valore aspettato, e il valore attuale, se coincidono allora il test va a buon fine.

- assertTrue(condition)
- assertNull(object)

Le notazioni fondamentali come, before each e before all servono rispettivamente per eseguire il metodo prima di ogni test, ed eseguire un metodo ogni volta prima di ogni test.

L'annotazione @ParameterizedTest permette di eseguire un metodo più volte passando in input valori diversi, senza inserire i valori nel codice di test, un modo per fornire i valori è tramite @CsvSource

COPERTURA DEL CODICE-> La copertura del codice è il rapporto che c'è tra le linee di codice eseguite e le linee di codice totali. Quando i metodi hanno più di un istruzione, dobbiamo capire quale parte di codice stiamo andando ad eseguire con quel test. Il risultato del rapporto descritto prima ci dice le linee di codice che abbiamo eseguito rispetto al totale, più la percentuale è alta e meglio è, dato che tutte le linee di codice sarebbero coperte.

REFACTORING->

Il refactoring è una tecnica che viene utilizzata su una progettazione software che permette di modificare la sintassi di un codice senza modificarne l'aspetto esterno di esso, e quindi la funzionalità. La progettazione si migliora durante e dopo la progettazione del codice, in modo tale da poter rilevare bug o frammenti di codice duplicati. Il refactoring nella maggior parte dei casi viene fatto per migliorare la progettazione del codice. Il refactoring viene fatto in diversi passaggi:

- Verificare il codice "sporco" con dei test, in modo tale da verificare se funzionino per bene;
- Utilizzare varie tecniche del refactoring;
- Verificare nuovamente con dei test il codice dopo aver fatto refactoring per vedere se sono superati;
- Se non sono superati allora vuol dire che il refactoring è andato ad intaccare qualcosa di sbagliato nel codice;

L'idea del refactoring, è anche la consapevolezza di riconoscere che un codice non verrà mai buono al primo tentativo, e quindi si devono fare varie riprogettazioni. Grazie al refactoring il codice diventa più corto e più facile da mantenere. La sintassi diventa più semplice e si evita di introdurre un **debito tecnico** nella progettazione.

PERCHÈ FARE REFACTORING?->

Come abbiamo già affrontato precedentemente, se un sistema software funziona ed è molto richiesto, avrà sempre bisogno di aggiornamenti che gli permettano di portare novità, ma questi aggiornamenti deteriorano il codice man mano che vengono fatti, e per questo deve essere fatto il refactoring. Oltre che togliere il codice duplicato, aiuta anche a rilevare i bug.

QUANDO FARE REFACTORING?->

Il refactoring viene fatto quando nel codice ci sono alcuni sintomi(code smell), e in base a questo usiamo la tecnica migliore tra cui:

- *Codice duplicato*: usiamo la tecnica Estrai Metodo
- *Metodi lunghi(o con commenti all'interno)*: usiamo la tecnica Estrai Metodo
- *Classi di grandi dimensioni*: usiamo la tecnica Estrai Classe

- *Lunga lista di parametri*: Introduciamo un oggetto parametro

Tecnica (1) Estrai Metodo

- Un frammento di codice può essere raggruppato. Far diventare quel frammento un metodo il cui nome spiega lo scopo del frammento

```
public void printDovuto(double somma) {  
    printBanner();  
    System.out.println("nome: " + nome);  
    System.out.println("tot: " + somma);  
}
```

- Diventa

```
public void printConto(double somma) {  
    printBanner();  
    printDettagli(somma);  
}  
  
public void printDettagli(double somma) {  
    System.out.println("nome: " + nome);  
    System.out.println("tot: " + somma);  
}
```

Con questa tecnica si vanno a creare metodi più piccoli e funzionali che possono essere richiamati da altri metodi, ma soprattutto che abbiano un nome che li descriva con aggiunta anche di commenti. Con un abbondanza di metodi piccoli, i metodi più grandi che poi richiameranno questi metodi più piccoli sembreranno una sequenza di commenti. Per le variabili avremo che se sono locali nel metodo sorgente allora potranno essere chiamate in input nel metodo, altrimenti se usate solo nel frammento di codice potranno essere impostate come variabili temporanee.

MECCANISMI:

Di seguito gli step da seguire per applicare questa tecnica

1. Creare un nuovo metodo il cui nome comunica l'intenzione del metodo;
2. Copiare il codice estratto;
3. Guardare se il codice estratto ha variabili locali nel metodo sorgente e nel caso farli diventare parametri del nuovo metodo;
4. Se alcune variabili sono locali del nuovo metodo farle diventare variabili temporanee;
5. Se il codice estratto modifica una variabile locale del metodo originario, dobbiamo vedere se è possibile mettere come valore di ritorno della funzione estratta quel valore. (Metodo sostituisci temp con query);
6. Sostituire nel metodo sorgente il codice estratto con una chiamata al metodo nuovo;

Per applicare estrai metodo, bisogna prima cercare un metodo lungo, o un codice che senza commenti diventa difficile da capire. Dividere il metodo lungo in metodi piccoli e assegnando ad ognuno di tali metodi un nome che descrive cosa quel metodo fa. Questi commenti piccoli vengono poi richiamati dai commenti grandi, che sembreranno più puliti e sembreranno una sequenza di commenti.

SOSTITUIRE UNA TEMP CON QUERY->

Una variabile temporanea è usata per tenere il risultato di un espressione, per migliorare

questa situazione dobbiamo:

- Estrarre l'espressione ed inserirla in un metodo;
- Sostituire tutti i riferimenti a temp con la chiamata al metodo che incapsula l'espressione;
- Il nuovo metodo può essere richiamato anche altrove !

Questo metodo di sostituzione di una temp con una query non fa altro che risolvere il problema della variabile temporanea in un metodo, ossia che invece che creare vari variabili temporanee, possiamo creare un metodo(query) che restituisce lo stesso valore che era contenuto dalla variabile temporanea, così che possiamo richiamarla sempre tramite la chiamata del nome del metodo nella quale è contenuto.

Le motivazioni sono date dal fatto che le temp sono variabili temporanee visibili solo dentro quel metodo, e quindi per andarle a prendere allungano i metodi. Con la sostituzione effettuata tramite il refactoring, qualsiasi metodo può avere il dato. Quindi prima si sostituisce temp con query e poi si fa la tecnica dell'estrai metodo.

I meccanismi per fare questa sostituzione sono:

- Cercare una variabile temporanea assegnata solo una volta
- Dichiarare temp final
- Compilare (per verificare che è assegnata una volta sola)
- Estrarre la parte destra dell'assegnazione e creare un metodo

Esempio Sostituisci Temp Con Query

```
private double quantita, prezzo;  
  
public double getPrezzo1() {  
    double prezzoBase = quantita * prezzo;  
    double sconto;  
    if (prezzoBase > 1000) sconto = 0.95;  
    else sconto = 0.98;  
    return prezzoBase * sconto;  
}
```

- Diventa, sostituendo entrambe le variabili prezzoBase e sconto

```
private double quantita, prezzo;  
  
public double getPrezzo2() {  
    return prezzoBase() * sconto();  
}  
  
private double prezzoBase() {  
    return quantita * prezzo;  
}  
  
private double sconto() {  
    if (prezzoBase() > 1000) return 0.95;  
    return 0.98;  
}
```

```
double temp = 2*(height+width); // temp è il perimetro del rettangolo
temp = height*width;           // Adesso temp è l'area del rettangolo

// Per avere nomi più appropriati ed avere un codice più leggibile diventa
...
final double perimeter = 2*(height+width);
final double area = height*width;
```

La temp in un metodo dovrebbe essere assegnata una sola volta, perché assegnare una temp a 2 valori diversi confonde il lettore, tranne che ad esempio sia una temp che cambia valore ad ogni ciclo, allora in quel caso va bene. Quello che dovremmo fare è usare una variabile separata per ogni assegnamento. Perdiamo di efficienza ma il codice è più pulito. La motivazione è: Se le variabili hanno più di una responsabilità all'interno di una funzione non va bene, una variabile temporanea deve avere una singola responsabilità, perché usare la stessa variabile per cose diverse confonde il lettore. Vediamo la differenza nell'immagine sopra, nella prima parte abbiamo la stessa variabile temporanea che contiene 2 cose differenti, invece sotto abbiamo l'assegnazione di 2 diverse variabili, che anche se occupano più memoria rendono il codice più pulito e meno rischioso.

SPOSTA METODO->

Un metodo usa più caratteristiche (attributi e operazioni) di un'altra classe che non di quella in cui è definito. Quello che si dovrebbe fare è che se un metodo usa attributi di una classe che non è la sua allora nell'altra classe si deve andare a creare un metodo con il corpo simile al primo metodo creato, e poi nel metodo vecchio si richiama questo metodo nuovo. Significa che se un metodo utilizza più metodi di un'altra classe rispetto a quella in cui è stata prodotta allora deve essere spostato. Uno dei vantaggi non è solo che gli attributi di quel metodo sono coerenti con quella classe in cui è stato messo, ma soprattutto c'è meno interazione tra le classi, e quindi c'è più modularità.

Motivazioni->

- Due classi sono strettamente accoppiate
- Spostare un metodo rende le responsabilità delle classi più chiare
- Un metodo usa i metodi di un'altra classe più di quanto usa i metodi della propria classe

Meccanismi->

- Esaminare attributi e metodi usati dal metodo da spostare per decidere se spostare anche questi (spostarli se sono usati solo dal metodo trasferito).
- Controllare che il metodo da spostare non sia dichiarato anche in superclassi e sottoclassi.
- Dichiarare il metodo nella classe in cui vogliamo che sia messo;
- Decidere se rimuovere il metodo originario o tenerlo per delegare (chiamare al suo interno il nuovo metodo nell'altra classe)

ESTRAI CLASSE->

- Una classe fa il lavoro che dovrebbero fare due classi
- Es. la classe Persona ha attributi per tenere prefisso e numero di telefono

- Si crea una nuova classe e si spostano alcuni attributi e metodi dalla vecchia alla nuova classe

Motivazione ->

- Una classe dovrebbe avere una responsabilità, ma durante lo sviluppo si aggiungono attributi e metodi e la classe cresce
- Una classe grande è difficile da comprendere
- Se un sotto-insieme di dati ed un sotto-insieme di metodi dovrebbero andare insieme, o se un sotto-insieme di dati di solito viene cambiato insieme, o tali dati dipendono tra loro, allora è bene dividere la classe
- Un altro segno è che la sottoclasse di tale classe coinvolge solo poche caratteristiche della classe

La tecnica consiste nel **prendere una singola classe che sta facendo il lavoro di due (o più) classi, e dividerla**. Si crea una nuova classe e si spostano i campi e i metodi pertinenti dalla vecchia classe a quella nuova.

Il principio guida qui è il **Single Responsibility Principle (SRP)**: una classe dovrebbe avere uno e un solo motivo per cambiare.

Devi applicare questa tecnica quando ti imbatti in un *Code Smell* noto come **Large Class** (Classe Enorme) o quando noti un gruppo di dati che stanno sempre insieme.

I sintomi classici sono:

- La classe ha troppi campi, troppi metodi e troppe righe di codice.
- Un sottoinsieme di campi e metodi inizia spesso con lo stesso prefisso (es. `indirizzoVia`, `indirizzoCitta`, `indirizzoCap`).
- Quando devi fare una modifica al sistema, ti trovi a toccare questa classe per motivi completamente diversi (es. devi aggiornare la logica di calcolo del carrello *oppure* le regole di formattazione del profilo utente, ed entrambe le cose sono nella stessa classe).

CLASSI ED EREDITARIETA'->

Quando vogliamo che una classe venga estesa quindi che si vogliano usare classi esistenti per metodi e attributi diversi, non è pratico copiare la classe originaria e modificarne gli attributi o metodi. Il riuso delle classi deve avvenire senza duplicazione del codice ma soprattutto in modo semplice per il programmatore. Un modo semplice è ovviamente l'ereditarietà tra classi, quindi avere una classe madre e una classe figlia che eredita gli oggetti da quella madre.

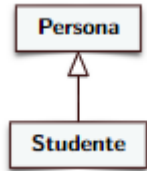
L'ereditarietà tra classi è un modo molto efficiente che esiste nella programmazione software, che permette di riscrivere un'altra classe che dipende da un'altra, e aggiungere gli attributi in più che la classe madre non ha. Ad esempio se avessimo la classe persona come classe madre, avente gli attributi(nome,cognome), potremmo mettere una classe figlia chiamata studente, che quando verrà creata automaticamente avrà tutti gli attributi e metodi della madre, e poi potremmo aggiungere quelli suoi personali come(matricola, voti ecc..). La

sintassi viene scritta così:

La classe `Studente` eredita da `Persona`:

```
public class Studente extends Persona
```

Nei diagrammi UML l'ereditarietà tra classi viene rappresentata tramite una freccia, che ha una forma ben precisa, ossia una freccia con un triangolo chiuso nel verso della classe madre, quindi nel caso descritto prima tra `Persona` e `Studente` avremo:



La sottoclasse ereditando metodi e attributi della superclasse può usarli come se fossero locali, e quindi interni ad esso. C'è un caso in cui abbiamo 2 metodi con lo stesso nome in entrambe le classi, in quel caso noi creeremo un'istanza della classe rispettiva, e poi chiameremo il metodo con la rispettiva istanza, quindi in base all'istanza con cui chiamiamo il metodo che ha il medesimo nome in entrambe le classi, chiameremo quello specifico metodo appartenente alla classe a cui appartiene l'istanza.

RECORD->

un `record` è una classe speciale, cortissima e immutabile, progettata con l'unico scopo di contenere e trasportare dati. Una volta scritto un record, i dati al suo interno saranno immutabili e quindi non potrai più modificarli ma solo leggerli.

Un record è una struttura dati che può contenere dati immutabili, ciò significa che non possono essere modificati. Il record viene dichiarato con `final`, ossia che non può essere ereditato da altre sottoclassi. Può contenere metodi e attributi static, e può contenere anche interfacce.

La sintassi del record è:

```
public record Esame(String materia, int voto) { }
```

Tra le parentesi graffe vediamo che non ci sono i puntini puntini, dato che all'interno di questo record non dobbiamo mettere niente, dobbiamo semplicemente mettere degli attributi in input nella funzione, così che vengano creati dal compilatore 2 metodi che servano per leggere questi 2 attributi.

I record vengono usati per sostituire tutti i metodi inutili che vengono fatti in una classe, questo significa che in quella classe non bisognerà più mettere il costruttore o vari metodi per stampare nome, cognome ecc. come nel caso di `studente`, ma lui stesso creerà dei metodi (sotto al cofano di java) che permetteranno di evitare l'implementazione di svariate righe di codice per una cosa così banale.

Invece di scrivere 50 righe di classe `Studente` con getter, costruttori, ecc., scrivi solo questo:

Java

```
// Un Record che contiene nome, cognome e matricola
public record Studente(String nome, String cognome, int matricola) {}
```

Fine. Java ha già creato tutto. Ora puoi usarlo così:

Java

```
public class Main {
    public static void main(String[] args) {
        // Creiamo lo studente
        Studente s1 = new Studente("Mario", "Rossi", 12345);

        // Leggiamo i dati (Nota: non si usa getName(), ma direttamente nome())
        System.out.println("Il nome è: " + s1.nome());

        // Stampa automaticamente una stringa leggibile (grazie al toString automatico)
        System.out.println(s1); // Stampa: Studente[nome=Mario, cognome=Rossi, matricola=12345]
    }
}
```

EREDITARIETA'->

La visibilità di metodi e attributi, sono sempre gli stessi, ossia `protected`, `private` e `public`. Con l'ereditarietà otteniamo `protected` che significa che sia visibile sia nella classe padre che in quella figlia ma non all'esterno.

INTERFACCE->

Le interfacce sono pura astrazione, e rispetto ai record dicono cosa una classe deve fare ma non dicono come deve farlo. Quindi nell'interfaccia scrivo cosa quella classe deve fare, e poi tramite la parola "implements" faccio una classe dove implemento quello che ho scritto nell'interfaccia. Per questo l'interfaccia viene considerato un concetto astratto, perché lascia decidere a te come fare le cose. Un'interfaccia quindi non è altro che una collezione di metodi vuoti (senza corpo), che poi dovranno essere implementati dalla classe che sceglierà di "firmare" con tale interfaccia. Serve a creare "disaccoppiamento". Se il tuo programma si aspetta di lavorare con qualcosa che sa "Stamparsi", a lui non importa se quel qualcosa è un Documento PDF, una Fotografia o un File di Testo. Gli basta sapere che obbedisce al contratto "Stampabile". Questo rende il codice facilissimo da espandere in futuro.

Nel diagramma UML l'interfaccia come abbiamo già visto viene implementata con il rettangolo con dentro scritto `<< interface` e per collegare le frecce nel diagramma usiamo delle frecce tratteggiate che indicano proprio l'interfaccia, e all'estremità il triangolo vuoto rivolto verso l'interfaccia.

L'annotazione (`@Override`) è definita nelle librerie standard di Java, e si deve mettere vicino ad un metodo, e serve per comunicare al compilatore che stiamo volendo fare la

ridefinizione del metodo della superclasse, già gliela diamo con il nome del metodo e la lista dei parametri, quindi la sintassi dell'override è qualcosa in più che può evitare di commettere qualche errore. La sintassi super ci fa richiamare un metodo della superclasse in una classe figlia.

Ecco come funziona l'interfaccia:

Definiamo il contratto:

Java

```
// Questa è l'interfaccia. Dice solo COSA fare.  
public interface Emettibile {  
    void emettiSuono();  
}
```

Ora creiamo due classi diverse che firmano questo contratto:

Java

```
public class Automobile implements Emettibile {  
    @Override  
    public void emettiSuono() {  
        System.out.println("Beep Beep!"); // Ecco COME lo fa l'auto  
    }  
}  
  
public class Gatto implements Emettibile {  
    @Override  
    public void emettiSuono() {  
        System.out.println("Miao!"); // Ecco COME lo fa il gatto  
    }  
}
```

CLASSI ASTRATTE->

Le classi astratte sono delle classi diverse da quelle normali, dato che implementano al suo interno metodi che sono astratti. Questo implica che quando andiamo ad implicare una classe astratta e al suo interno i metodi astratti, andremo a mettere la parolina magica **abstract** che ci permette di rendere tali metodi e classi astratte. Il senso della classe astratta è quello di utilizzare i metodi ma non implementarli, questo è utile per utilizzarli successivamente. **Il senso sarebbe quello di utilizzare tali metodi come stampo per le sottoclassi, ma soprattutto per costringere tali sottoclassi ad implementare tali metodi.** Ovviamente tutto questo avviene con l'ereditarietà che ricordiamo è una cosa unidirezionale ossia che la classe che sta più in alt nella gerarchia mi permette di stabilire un'istanza di classi che stanno al di sotto di essa, e non si può fare viceversa.

- La classe astratta **non** può essere istanziata

```
public abstract class Libro {
    private String autore;
    protected List<Pagina> pagine;
    public abstract void inserisci(Pagina p);
    public String autore() {
        return autore;
    }
}
```

8

<i>Libro</i>
-autore:String -pagine:List<Pagina>
+ <i>inserisci(p:Pagina)</i> +autore():String

Prof. Tramontana - Marzo 2025

Ovviamente alcuni metodi della classe astratta possono essere implementati. Una classe astratta come si legge sopra non può essere istanziata.

Fatta la classe astratta ovviamente tali metodi dovranno essere implementati dalle classi figlie. Questo viene fatto per rispettare uno dei concetti più importante della programmazione ad oggetti, ossia il DRY(don't repeat yourself) e quindi invece che magari riscrivere tantissime volte le stesse informazioni per ogni oggetto, verranno implementate ognuna dalla propria classe figlio che erediterà tutti i metodi(astratti e non) della classe. I metodi astratti poi dovranno essere implementati con @Override.

Java

```
// La parola chiave 'abstract' indica che è un modello incompleto
public abstract class Lavoratore {

    // 1. Dati condivisi da tutti i lavoratori
    protected String nome;

    // Costruttore
    public Lavoratore(String nome) {
        this.nome = nome;
    }

    // 2. Metodo CONCRETO (comune a tutti, già scritto)
    public void timbraCartellino() {
        System.out.println(nome + " ha timbrato alle 09:00.");
    }

    // 3. Metodo ASTRATTO (manca il codice: ogni figlio lo farà a modo suo)
    public abstract double calcolaStipendio();
}
```

```

Java

// Impiegato EREDITA da Lavoratore
public class Impiegato extends Lavoratore {

    public Impiegato(String nome) {
        super(nome); // Chiama il costruttore del padre
    }

    // Obbligatorio: implementiamo il metodo astratto
    @Override
    public double calcolaStipendio() {
        return 1500.00; // Stipendio fisso
    }
}

// Freelance EREDITA da Lavoratore
public class Freelance extends Lavoratore {
    private int oreLavorate;
    private double tariffaOraria;

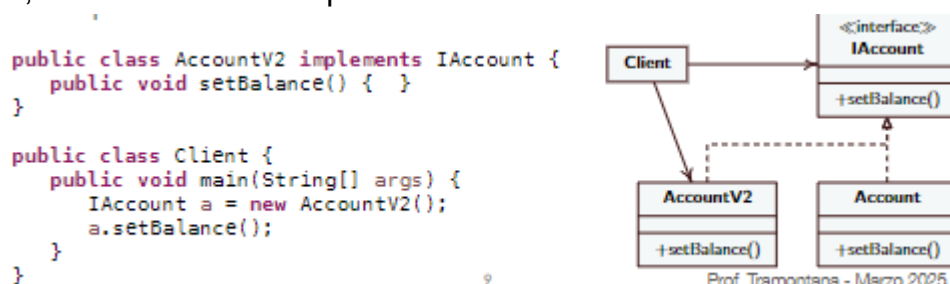
    public Freelance(String nome, int oreLavorate, double tariffaOraria) {
        super(nome);
        this.oreLavorate = oreLavorate;
        this.tariffaOraria = tariffaOraria;
    }

    // Obbligatorio: implementiamo il metodo astratto a modo nostro
    @Override
    public double calcolaStipendio() {
        return oreLavorate * tariffaOraria; // Stipendio variabile
    }
}

```

CLASSI E INTERFACCE->

Come abbiamo detto precedentemente una classe può implementare i metodi che vengono definiti dall'interfaccia. I client che sono le classi che utilizzano i metodi implementati e non dell'interfaccia sanno cosa possono invocare e cosa possono usare. Anche se un'interfaccia muta, il client che utilizza quell'interfaccia rimane invariato.



Quello a destra è l'implementazione della correlazione tra classe e interfaccia nel diagramma UML.

COMPATIBILITA' FRA TIPI->

Sappiamo che la classe **Studiante** che eredita da **Persona**, quindi una sottoclasse che eredita dalla superclasse, prende tutti gli attributi e metodi che la classe **persona** ha, quindi nome, cognome, indirizzo ecc.. ma la classe **studiante** può anche implementare i suoi metodi personali come voto, matricola ecc.. . Una sottoclasse può prendere il posto di una

superclasse, questo significa che quando il codice richiede un oggetto di tipo persona tu puoi passargli tranquillamente un'oggetto di tipo studente.

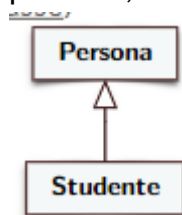
```
Java

// La "scatola" (variabile) è etichettata come Persona.
// Ma l'oggetto REALE che creiamo e ci mettiamo dentro è uno Studente!
Persona p = new Studente("Mario", "Rossi", 12345);

// Questo funziona perfettamente.
// Java cercherà il metodo printAll() dentro lo Studente.
p.printAll();
```

In questo caso come viene scritto nel codice ad un'istanza di persona(superclasse) viene affidato un'oggetto di tipo studente, e quindi i metodi richiamati con l'istanza di persona(p) faranno riferimento allo studente e al suo metodo printAll(). Infatti se ci fossero 2 metodi printAll() uno nella superclasse e uno nella sottoclasse, verrà richiamato quello che viene assegnato all'istanza di persona, e in questo caso come già detto sarà lo studente. Questa tecnica viene chiamata **Upcasting**.

Ovviamente non vale il viceversa, significa che l'Upcasting ovviamente vale da sotto verso sopra quindi sale verso su, e proprio come nel diagramma UML quindi uno studente è una persona, ma non tutte le persone sono studenti.



Quindi un codice scritto così:

```
Java

// ERRORE DI COMPILAZIONE! Java ti ferma subito.
Studente s = new Persona("Luigi", "Verdi");
```

È un errore che java ti fa notare subito.

Quando la sottoclasse eredita e usa i metodi della superclasse fa un override ossia che li modifica e li adatta per lui.

RAPPRESENTAZIONE DI EREDITARIETA' NEI DIAGRAMMI UML->

- Esempio di creazione di un'istanza della classe Studente

```
public class MainEsami {  
    public static void main(String[] args) {  
        Studente s = new Studente("Alan", "Rossi", "M12345");  
    }  
}
```

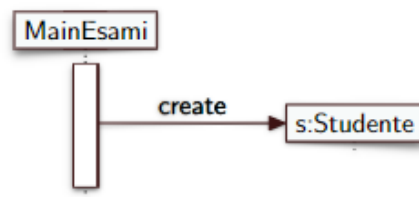
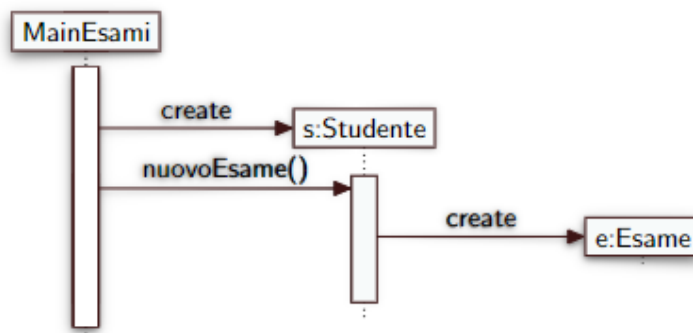


Diagramma UML Di Sequenza

- Esempio di chiamate di metodo a cascata

```
public class MainEsami {  
    public static void main(String[] args) {  
        Studente s = new Studente("Alan", "Rossi", "M12345");  
        s.nuovoEsame("Italiano", 8);  
    }  
}
```



14

Prof. Tramontana - Marzo 2025

Qui vediamo come si rappresentano le varie creazioni di istanze di classi e di metodi che poi potranno essere richiamati. Il diagramma UML quindi ci permette di rappresentare queste informazioni in tale modo. IL DIAGRAMMA UML NORMALE spiega il funzionamento ma non cosa accade nel tempo, ciò significa che non viene indicato come (ovviamente) nel codice, la sequenza di operazioni una dopo l'altra, ma semplicemente vengono messe nella "linea temporale" per indicare cosa avviene nel codice. Invece IL DIAGRAMMA UML DI SEQUENZA ci indica tramite una linea temporale cosa accade nel tempo.

Il rettangolo vuoto che collega tutti i quadrati dove all'interno ci sono le classi o le istanze o i metodi viene detta barra d'attivazione. La linea tratteggiata che c'è tra la barra d'attivazione e la classe (MainEsami) è detta linea della vita. Sopra la freccia che collega la barra d'attivazione con un'istanza o un attributo viene messa una piccola descrizione di quello che sta avvenendo, quindi un verbo oppure viene scritto il metodo che si sta chiamando. L'asse temporale ovviamente viene inteso in verticale verso il basso, quindi i fatti accadono in ordine dall'alto verso il basso. In orizzontale in alto vengono messi gli oggetti. La linea della vita parte dalla barra di attivazione verso l'oggetto se e solo se quell'oggetto esiste.

LATE BINDING E POLIMORFISMO->

Quello che abbiamo detto precedentemente introduce il concetto del binding dinamico, ossia che quando andiamo ad assegnare ad una istanza di tipo persona un'istanza di tipo studente, l'istanza chiamata sarà proprio quella di studente perché è l'unica che esiste, ed è anche coerente al concetto di Upcasting. Se ci fossero 2 istanze create, ossia una per studente e una per persona allora la si dovrebbe decidere a quale metodo fare chiamare il metodo specifico della classe, come in questo caso che viene fatto con un ciclo:

Late Binding E

```
public void m() {  
    Persona p = new Persona();  
    Studente s = new Studente();  
    Persona px;  
    if (i > 10)  
        px = p;  
    else  
        px = s;  
    px.printAll();  
}
```

Per quanto riguarda il **polimorfismo** invece qui riguarda l'utilizzo di una certa risorsa per più specifici compiti. Ad esempio in programmazione 2 abbiamo imparato ad usare il polimorfismo con i template, ma in questo caso dato che i template non ci sono, riguarda il fatto dell'utilizzo di una funzione chiamata allo stesso modo da 2 classi ereditarie tra loro. Questo fa in modo che per riconoscere quale classe richiama tale metodo, serve capire gli attributi che ci sono all'interno di quel metodo, e in base a quegli attributi di diverso tipo si capisce da chi è stata chiamata tale funzione. Nei sistemi ad oggetti possono esistere metodi che hanno la stessa signature e lo stesso nome ma in classi diverse. Come già detto prima quando ci sono più classi ereditarie allora si avranno metodi con lo stesso nome, e chiamare un metodo può avere effetti diversi.

```
Studente s;  
// ...  
s.nuovoEsame("Maths", 8);  
s.printAll();
```

• La variabile s potrebbe tenere il riferimento ad un'istanza di Studente o di StudenteLavor, quale metodo nuovoEsame() sarà chiamato è deciso a runtime, in base all'istanza. Lo stesso vale quando si ha una variabile di tipo Persona e per il metodo printAll()

Il concetto di "Tipo" vs "Istanza"

- **A Compile Time (quando scrivi il codice):** Il compilatore guarda solo il **tipo dichiarato** della variabile per decidere quali metodi puoi chiamare. Se dichiaro `Persona p`, potrai chiamare solo i metodi presenti in `Persona`.
- **A Runtime (quando il programma gira):** Java guarda l'**istanza reale** che la variabile sta puntando in quel momento. Se `p` punta a uno `Studente`, Java userà i metodi

specifici dello studente.

2. Il Dispatch dei Metodi

Il **dispatch** è il processo con cui Java cerca il metodo corretto da eseguire. Segue una gerarchia dal basso verso l'alto:

1. Controlla la classe dell'oggetto a runtime.
2. Se il metodo è definito lì, lo esegue.
3. Se non lo trova, sale alla superclasse e cerca lì, e così via.

Quindi quando si ha ad esempio una chiamata di metodi come in foto, avremo diversi modi di chiamate di variabili.

```
(la classe Studente è sottotipo di Persona)  
  
Persona p = new Studente();  
  
• Tramite cast forzo l'istanza p di tipo Persona sulla variabile s del sottotipo  
  Studente  
  
Studente s = (Studente) p;
```

In questo caso chiamando `Persona p = new Studente()` si può benissimo fare dato che in base all'ereditarietà della sottoclasse studente con la classe persona ogni studente è considerabile una persona. Viceversa si potrebbe fare solamente con il cast, cioè dato che non ogni persona è uno studente, allora facciamo il cast che ci permette di rendere quella persona studente.

DESIGN PATTERN TEMPLATE METHOD->

Il design pattern con template method viene implementato tramite le classi astratte. Le classi astratte contengono metodi che possono essere implementati o no. Quando non sono implementati ogni figlio si implementa per se la funzione che non viene implementata nella classe astratta, invece quelli implementati sono uguali per tutti i figli. Il template method permette di definire alcuni passi di un algoritmo senza cambiare struttura dell'algoritmo, dato che ne viene definita la struttura di tale algoritmo, che rimarrà tale. Questa ridefinizione dei passi dell'algoritmo viene fatto tramite le **classi astratte**, che sono delle classi che non vengono implementate quando vengono dichiarate, ma poi nelle classi figlie, magari chi le implementa fa delle implementazioni diverse di un algoritmo che ha una determinata struttura. Il template method quindi è quello che definisce lo scheletro di questo algoritmo che richiama le funzioni astratte. Le **classi concrete** sono quelle che implementano le funzioni astratte.

```
public abstract class AbstractClass {  
    public void templateMethod() {  
        primitiveOperation1();  
        primitiveOperation2();  
    }  
    protected abstract void primitiveOperation1();  
    protected abstract void primitiveOperation2();  
}  
  
public class ConcreteClass extends AbstractClass {  
    @Override  
    protected void primitiveOperation1() {  
        // codice per l'operazione  
    }  
    @Override  
    protected void primitiveOperation2() {  
    }  
}
```

I metodi template portano ad avere una struttura di controllo invertita, che permette a una superclasse di chiamare le operazioni di una sottoclasse e non viceversa ("il principio

Hollywood", ovvero "non chiamarci, noi ti chiameremo"). Infatti vedi che nella superclasse(concrete class) nel metodo template ci sono le chiamate dei 2 metodi che derivano dalla sottoclasse.

PS-> La differenza tra classe astratta ed interfaccia: La differenza tra i 2 è che la classe astratta implementa sia metodi con solo prototipi, sia metodi implementati che valgono per tutte le classi derivanti da essa; Invece l'interfaccia inserisce solo metodi non implementati da implementare successivamente.

DESIGN PATTERN->

Sono strutture software(micro-architetture) che vengono architettate per contenere un piccolo numero di classi che implementano soluzioni per problemi ricorrenti. Tali micro-architetture descrivono oggetti e classi coinvolti, e le interazioni tra loro.

Esistono tanti cataloghi di design pattern, per vari contesti, come: Sistemi centralizzati, concorrenti, distribuiti, real-time, etc. Il design pattern presenta problemi di progettazione ricorrente, e ne presenta soluzioni collaudate, generica ma specializzabile. Aiutano i principianti ad agire come se fossero esperti. Evitano di re-inventare concetti e soluzioni riducendo il costo.

Un pattern nomina, astrae ed identifica gli aspetti chiave di un problema di progettazione, le classi e le istanze che partecipano, e di come collaborano tra loro. Ci sono 5 parti fondamentali che descrivono un pattern e il problema che affronta:

-Nome: Descrive il vero scopo del pattern, e permette di lavorare con un alto livello di astrazione;

-Intento: Descrive brevemente le funzionalità e lo scopo;

-Problema: Descrive il problema che il design pattern deve affrontare, e le condizioni necessarie per applicare una soluzione. Qui si parla anche di forze, che sono obbiettivi e vincoli spesso contrastanti che si incontrano in questo contesto es. "voglio che il codice sia super flessibile per modifiche future" ma allo stesso tempo "voglio che sia facilissimo da leggere oggi"). Queste forze si scontrano, e il pattern è proprio la soluzione che cerca di metterle in equilibrio.

-Soluzione: La soluzione è la descrizione delle classi del design pattern, che sono letteralmente la risoluzione del problema posto sopra;

-Conseguenze: Indicano quelle che sono risultati, vantaggi e svantaggi dell'utilizzo del design pattern.

ORGANIZZAZIONE DESIGN PATTERN->

I design pattern sono organizzati in base allo scopo, e sono come un manuale di istruzioni universale che ci permette di risolvere un qualsiasi problema di programmazione senza sbatterci troppo la testa. Ovviamente è usato per risparmiare tempo, dato che sai che il tuo programma non crollerà mai in futuro, ma soprattutto per essere flessibile, dato che se si vuole aggiungere una nuova funzione non si deve cancellare tutto il programma e rifarlo

d'accapo, ma semplicemente apportargli qualche modifica. Esistono vari gruppi di design pattern organizzati in base allo scopo:

-CREAZIONALI: I creazionali sono usati per creare istanze, e i vari design pattern all'interno di questa famiglia sono: Singleton, Factory Method, Abstract Factory, Builder, Prototype. Questi tipi di design pattern sono usati per creare oggetti in modo + flessibile, invece di inserire dovunque `new Oggetto()`. Quindi magari hai diversi oggetti da dover creare, ma non vai a decidere nel main quale tipo di oggetto creare, ma fai un'unica funzione che crea diversi oggetti, aggiungendo solo funzioni aggiuntive. Quindi come visto anche nell'esempio sotto, non ci interessa come quell'oggetto viene creato, quindi i vari passi, ma semplicemente al programma interessa avere l'oggetto. È meglio nei pattern creazionali avere molti piccoli oggetti ed incastrarli tra loro come lego, invece che avere moltissime sottoclassi con rigide turnazioni che poi diventeranno difficili da gestire. Ovviamente come già detto non ci interessa come un oggetto viene creato, ma semplicemente se dovessimo cambiare come un oggetto viene creato ci basterà modificare la fabbrica dove lo produciamo.

Per riassumere, i creazionali quindi si occupano di gestire al meglio la creazione degli oggetti. Il fatto importante riguarda la gestione della creazione, ciò significa che se si permettesse ad ogni singolo metodo di poter creare un nuovo oggetto con `new oggetto`, sarebbe un casino dato che sarebbe confusionario per il codice. Quindi si sfrutta l'utilizzo di un'unica "fabbrica", dove il programma non si sporca le mani a creare l'oggetto, ma semplicemente chiede l'oggetto pronto. La parola chiave per questo tipo di design pattern è: **creazione sicura**.

ES->

```
interface Pizza {
    void prepara();
}

// 2. I vari tipi di pizza
class Margherita implements Pizza {
    public void prepara() { System.out.println("Preparo base, pomodoro, mozzarella"); }
}
class Diavola implements Pizza {
    public void prepara() { System.out.println("Preparo base, pomodoro, salame piccante"); }
}

// 3. LA FABBRICA (Il Pattern vero e proprio)
class Pizzeria {
    // Questo è il "Factory Method". Crea gli oggetti per noi!
    public Pizza ordinaPizza(String tipo) {
        if (tipo.equals("Margherita")) {
            return new Margherita();
        } else if (tipo.equals("Diavola")) {
            return new Diavola();
        }
        return null;
    }
}
```



```
// 4. Il Cliente (Main)
public class Main {
    public static void main(String[] args) {
        Pizzeria daGino = new Pizzeria();

        // Il main NON usa mai "new Margherita()". Chiede alla fabbrica!
        Pizza laMiaPizza = daGino.ordinaPizza("Margherita");
        laMiaPizza.prepara();
    }
}
```

-STRUTTURALI: Riguardano la scelta della struttura, e servono a montare insieme classi diverse per farle lavorare in un'unica struttura più grande, e quindi formando una composizione. L'analogia che tiene è proprio quella di un adattatore per la presa, ossia che se si ha una presa americana, e un computer con attacco italiano, non cambi né computer, né presa, ma inserisci un adattatore in mezzo.

Per riassumere, i pattern strutturali fanno collaborare pezzi diversi per formare una struttura che funziona, quindi proprio come l'analogia della presa, senza modificare le 2 strutture da far coincidere.

```

// 1. Quello che il muro italiano si aspetta di ricevere
interface SpinaItaliana {
    void inserisciNellaPresaItaliana();
}

// 2. Il nostro computer comprato in America (INCOMPATIBILE)
class ComputerAmericano {
    public void inserisciNellaPresaAmericana() {
        System.out.println("Computer collegato alla presa americana a 110V.");
    }
}

// 3. L'ADATTATORE (Il Pattern vero e proprio)
// Dice al muro: "Tranquillo, sono una spina italiana!"
class AdattatoreSpina implements SpinaItaliana {
    private ComputerAmericano computer; // Nasconde il computer americano dentro

    public AdattatoreSpina(ComputerAmericano comp) {
        this.computer = comp;
    }

    // Traduce il comando italiano in un comando americano
    @Override
    public void inserisciNellaPresaItaliana() {
        System.out.println("L'adattatore converte la corrente da 220V a 110V...");
        computer.inserisciNellaPresaAmericana(); // Chiama il metodo originale
    }
}

// 4. Utilizzo (Main)
public class Main {
    public static void main(String[] args) {
        ComputerAmericano mioMac = new ComputerAmericano();

        // Creiamo l'adattatore e ci infiliamo il computer
        SpinaItaliana cavoAdattato = new AdattatoreSpina(mioMac);

        // Ora possiamo attaccarlo al muro italiano!
        cavoAdattato.inserisciNellaPresaItaliana();
    }
}

```

-COMPORTAMENTALI: Riguardano la scelta dell'incapsulamento degli algoritmi, quindi si occupano di come gli oggetti comunicano tra loro e di come si dividono i compiti. L'analogia sarebbe quella che si ha una ricetta che è fissa con una determinata struttura, ma in realtà tu puoi decidere ad esempio se la base della torta possa essere pasta o frolla senza cambiare struttura ricetta. Quindi è proprio come il programma che stavamo facendo sui social network, ossia che abbiamo sempre lo stesso metodo di accesso per i social, ossia ci sono i passaggi dell'autenticazione, del messaggio, della conferma ecc, ma nelle determinate classi ognuno implementa il proprio metodo d'accesso per come vuole, magari twitter usa username e password, invece LinkedIn usa un codice api.

Per riassumere i comportamentali sono quelli che vengono implementati con le classi astratte, così che possano avere metodi che magari sono uguali tra loro, e metodi che ognuno implementa per se in modo diverso, così che creando un'unica classe si possono

implementare diverse strutture.

```
// 1. LO SCHELETRO (Il Pattern vero e proprio)
abstract class SocialNetwork {

    // TEMPLATE METHOD: È segnato come "final" perché le figlie NON possono
    // cambiare l'ordine delle azioni. Il regolamento lo detto io!
    public final void pubblicaMessaggio() {
        login();
        inviaDati();
        logout();
    }

    // Metodi astratti: lo scheletro dice che DEVONO esistere,
    // ma lascia alle figlie il compito di scriverli.
    abstract void login();
    abstract void inviaDati();
    abstract void logout();
}

// 2. La classe figlia personalizza i dettagli
class Facebook extends SocialNetwork {
    @Override
    void login() { System.out.println("Login su Facebook con email e password."); }
    @Override
    void inviaDati() { System.out.println("Pubblico un post sulla bacheca di Facebook."); }
    @Override
    void logout() { System.out.println("Disconnessione da Facebook."); }
}

class Twitter extends SocialNetwork {
    @Override
    void login() { System.out.println("Login su Twitter con @username."); }
    @Override
    void inviaDati() { System.out.println("Pubblico un Tweet di 280 caratteri."); }
    @Override
    void logout() { System.out.println("Disconnessione da Twitter."); }
}

// 3. Utilizzo (Main)
public class Main {
    public static void main(String[] args) {
        SocialNetwork mioSocial = new Facebook();

        // Chiamo un SOLO metodo, e lui esegue tutto lo scheletro in automatico!
        mioSocial.pubblicaMessaggio();
    }
}
```

FACTORY METHOD(CREAZIONALE)->

L'intento del factory method è proprio quello di definire un'interfaccia per creare un oggetto, ma lasciare decidere alle sottoclassi quale classe istanziare, e quindi si rimanda l'istanziazione alle sottoclassi. L'esempio è proprio quello di un capo che nell'interfaccia dice che qui si fabbricano veicoli, ma lui non si sporca le mani in fabbrica, ma delega il lavoro ai reparti specializzati(sottoclassi) e sono loro che creeranno nuovi oggetti.

Il problema è che ovviamente se si devono creare diversi oggetti per vari casi, come ad esempio dei nemici per super mario, noi avremo che quando si genera un livello, in quel livello si dovrà inserire un nemico, ma si deve capire quale nemico inserire, e ovviamente non si devono conoscere tutti i nemici a memoria ed instanziarli volta per volta.

Per risolvere questo usiamo il factory method, che quindi consiste nella creazione di una sottoclasse che è specializzata nella creazione di nemici in base al livello in cui ci si trova.

Ci sono varie variabili che definiscono il factory method:

-Product-> è letteralmente la classe astratta, quindi è l'interfaccia comune degli oggetti creati, quindi nel nostro esempio sopra è la classe astratta nemico che ci dà l'idea generale, quindi ci dice cosa deve saper fare l'oggetto ma non come farlo, quindi nel caso di un'azienda di trasporti sarebbe l'idea di mezzo di trasporto che deve saper consegnare un pacco;

-ConcreteProduct-> è un'implementazione del product, quindi quello che esce dalle sottoclassi, e quindi il nemico specifico che viene implementato in una sottoclasse. È l'oggetto fisico e tangibile, ossia quello che esce dalla fabbrica e fa il lavoro, quindi nel caso di un'azienda di trasporti sarebbe il mezzo che svolge il lavoro di consegna pacchi, quindi camion, nave, aereo;

-Creator-> Il creator è il concetto generale di creazione di nemici, quindi implementandolo ritorna un'oggetto di tipo product, quindi poi implementandolo tramite il factory method sputa fuori un concrete product. Dice che chiunque lavori per lui deve avere un pulsante per fabbricare un product.

-ConcreteCreator-> Il concrete creator è la sottoclasse che estende la classe astratta, quindi da qui esce il nemico specifico. È il reparto specifico che prende la decisione finale. È lui che usa davvero il comando `new` per costruire l'oggetto giusto.

SINGLETON(CREAZIONALI)->

Intento: assicurare che una classe abbia una sola istanza e fornire un punto d'accesso globale all'istanza

Il singleton che letteralmente significa singolo, si assicura di fare l'opposto rispetto al factory method, ossia non creare tantissime istanze come l'esempio della fabbrica, ma si assicura di creare una sola istanza per tutto il programma. Quindi prendendo l'esempio del presidente della repubblica, il presidente in Italia è uno solo, e se più persone vogliono parlare con lui devono rivolgersi sempre alla stessa persona. La soluzione per creare questa unica istanza è quella di porre il costruttore in `private`, così che nessuno può creare una nuova istanza di presidente, e l'unica che viene creata rimarrà sempre l'unica nell'intero programma. Eventualmente se non viene creata lo verifichiamo e verrà creata.

Java

```
public class Presidente {  
  
    // 1. Qui dentro salviamo l'UNICA copia esistente  
    private static Presidente istanzaUnica;  
  
    // 2. IL TRUCCO: Costruttore PRIVATO. Nessuno da fuori può fare "new Presidente"  
    private Presidente() {  
        System.out.println("È stato eletto un nuovo Presidente!");  
    }  
  
    // 3. IL PUNTO DI ACCESSO: L'unico modo per avere il Presidente  
    public static Presidente getInstance() {  
        // Se non esiste ancora, lo creiamo per la prima e unica volta  
        if (istanzaUnica == null) {  
            istanzaUnica = new Presidente();  
        }  
        // Restituiamo l'unica copia esistente  
        return istanzaUnica;  
    }  
}
```

Se nel tuo `main` provi a fare questo:

Java

```
Presidente p1 = Presidente.getInstance();  
Presidente p2 = Presidente.getInstance();
```

Java non creerà due Presidenti. `p1` e `p2` punteranno esattamente allo stesso identico oggetto nella memoria.

Tutto chiaro? È letteralmente un lucchetto messo alla parolina `new` !

Per riassumere quindi, letteralmente l'istanza creata è l'unica perché nella classe in cui viene creata viene messo tutto privato, così che quando si proverà a creare nel main più istanze, esse punteranno allo stesso indirizzo di memoria.

Come variante del singleton vi è il MULTITON, che invece che avere una sola istanza possibile in tutto il programma, ha un numero limitato di istanze che si possono creare, che verificandolo con un while, possiamo limitarlo.

```
public class Fibonacci {  
    private static Fibonacci obj = new Fibonacci();  
    private static instanceCount = 0;  
    private int[] x = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144};  
    private Fibonacci(){}  
  
    public static Fibonacci getInstance(){  
        if(instanceCount <= 5){  
            obj = new Fibonacci();  
            instanceCount++;  
        }  
        return obj;  
    }  
}
```

OBJECT POOL->

L'object pool, o piscina di oggetti, è letteralmente un posto dove vengono riciclati gli oggetti del programma invece che di creare oggetti nuovi ogni volta che il programmatore lo chiede.

È la stessa filosofia di quando vengono distribuite le scarpe da bowling, ossia che quando vengono le persone, non gli vengono date scarpe nuove ad ognuno, ma semplicemente vengono riciclate sempre le stesse scarpe.

Quindi è un deposito di istanze già create, che vengono riciclate e riusate quando un client glielne chiede. Tale magazzino di oggetti può crescere o può avere dimensioni fisse. Nel caso delle dimensioni fisse ovviamente avremo una dimensione che non andrà ad aumentare, quindi se ci sono 50 paia di scarpe, ed entra il 51 cliente, esso dovrà aspettare che qualcuno restituisca le scarpe. Quindi il client quando avrà finito con tale oggetto lo restituirà al pool che lo rimetterà a disposizione per gli altri client. Tutto questo si può collegare al factory method, perché si riciclano tali oggetti, ma se l'object pool può crescere invece che riciclare, se un client richiede ma non solo disponibili, se ne può creare uno nuovo.

Ovviamente quando un oggetto viene riciclato quell'oggetto deve essere resettato, perché senno quel client che lo deve usare si trova dentro i dati del client che l'ha usato prima, quindi nel caso delle scarpe, si deve spruzzare il deodorante dentro prima che l'altro cliente le usi. Ogni client dovrà indicare quando quell'oggetto non lo sta usando più ed è riusabile. Il magazzino object pool nel programma deve essere unico, quindi si usa un singolo che garantisce che esista una sola copia di quella classe.

Esempio Di Object Pool

```
import java.util.ArrayList;
import java.util.List;

// CreatorPool è un ConcreteCreator e implementa un Object Pool
public class CreatorPool {
    private List<Shape> pool = new ArrayList<>();

    // metodo factory che ritorna un oggetto prelevato dal pool
    public Shape getShape() {
        if (pool.size() > 0)
            return pool.remove(0);
        return new Circle();
    }

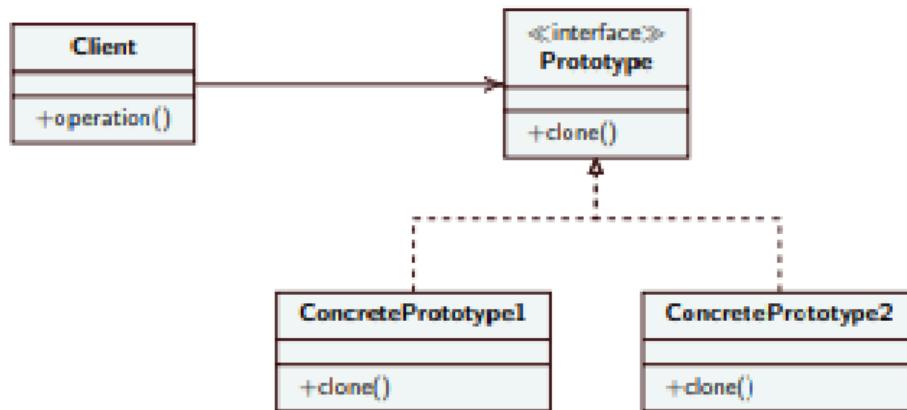
    // inserisce un oggetto nel pool
    public void releaseShape(Shape s) {
        pool.add(s);
    }
}
```

In questo codice vediamo come abbiamo un array list che funge da piscina per gli oggetti, abbiamo il codice che inizializza tale array list che all'inizio sarà vuoto. Poi si fa un metodo che verifica prima se quell'array è vuoto, se non lo è allora rimuove un oggetto da quell'array list, altrimenti forma un nuovo oggetto e lo inserisce nell'array list.

PROTOTYPE(CREAZIONALI)->

Questo design pattern è il più semplice dato che funge come una fotocopiatrice che parte da un prototipo iniziale. Mettiamo che dobbiamo creare 100 oggetti uguali, se dovessimo fare 100 volte la creazione magari di un oggetto difficile da creare, la fatica sarebbe molto elevata, ma quello che dobbiamo fare noi con questo design pattern è fare la fatica una sola volta, così che poi cloniamo questo oggetto per crearne uno nuovo identico risparmiando la

fatica.



-**Client:** È il tuo programma principale. Lui ha in mano un oggetto e dice semplicemente: *"Fammi una copia di questo"* chiamando il metodo `clone()` .

-**Prototype (Interfaccia):** È il "patentino" che dice: *"Tutti gli oggetti che vogliono essere fotocopiables devono avere un pulsante chiamato `clone()` "*.

-**ConcretePrototype1 e 2:** Sono gli oggetti veri e propri (es. l'Orco e il Mago) che implementano il pulsante e sanno materialmente come fare una copia esatta di se stessi.

Quindi rispetto al factory method qui non crea un oggetto nuovo di zecca da zero, ma crea semplicemente il clone identico dell'oggetto già esistente.

Sappiamo che il client non conosce quale oggetto si sta clonando, ma conosce semplicemente l'interfaccia dove c'è il comando clone, quindi non è a conoscenza dell'oggetto che lui sta clonando.

Esiste un manager di prototipi che è come se fosse una vetrina dove viene messo l'oggetto che deve essere clonato, e quindi non tieni gli originali sparsi a caso nel codice.

Distinguiamo 2 livelli di clonazione:

Copia Superficiale (Shallow copy):

- **Cosa dice la slide:** Vengono tenuti i riferimenti degli attributi, quindi clone e prototipo sono strettamente legati. Spesso questo non basta.
- **Cosa significa:** Immagina di clonare un Orco che ha in mano una clava. Con la copia superficiale, Java clona il corpo dell'Orco, ma per risparmiare memoria **non clona la clava**. Ora hai due Orchi che condividono *la stessa identica clava magica*. Se il clone rompe la clava, sparisce anche all'Orco originale! È un disastro.

Copia Profonda (Deep copy):

- **Cosa dice la slide:** Clone e prototipo sono totalmente separati nei loro attributi. I cloni vengono istanziati impostando i dati senza mantenere un riferimento all'originale.
- **Cosa significa:** Fai le cose per bene. Cloni il corpo dell'Orco E gli forgi una clava tutta sua, nuova di zecca. Ora i due Orchi sono completamente indipendenti al 100%. Se uno

muore o perde le armi, l'altro non ne risente minimamente.

```
public interface Prototype {  
    Prototype clone();  
    void stampa();  
}
```

```
public class CompactModule implements Prototype{  
    private final int id;
```

```
    private final int mode;  
    private final String name;  
  
    public CompactModule(int id, int mode, String name){  
        this.id=id;  
        this.mode=mode;  
        this.name = name;  
    }  
  
    public Prototype clone() { return new CompactModule(id, mode, name);  
  
    public void stampa(){ ... }  
}
```

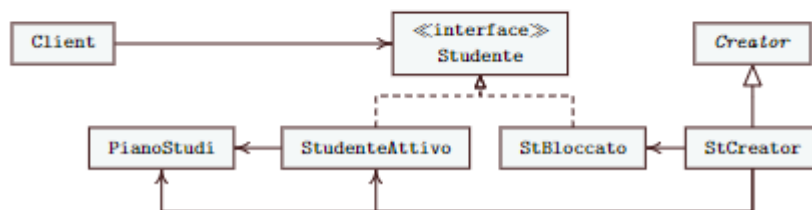
```
public class Client {  
    public static void main(String[] args){  
        Prototype firstProto = new CompactModule(8, 23, "CompactModule");  
        firstProto.stampa();  
        Prototype nuovaCopia = firstProto.clone();  
        nuovaCopia.stampa();  
    }  
}
```

DEPENDENCY INJECTION->

Un oggetto non si deve mai costruire le cose che gli servono(da cui dipende) da solo, ma deve sempre farsele consegnare già pronte. Tali oggetti da cui dipende, vengono consegnate ad esso nel suo costruttore, dato che stiamo sempre parlando di sottoclassi che ereditano da una classe astratta. Tale classe non deve sapere come è stato creato tale oggetto ma semplicemente deve riceverlo e basta.

Esempio Di Dependency Injection

- Si abbia una classe `PianoStudi` che è una dipendenza per `Studente`
- `Studente` riceve nel suo costruttore l'istanza di `PianoStudi`
- `Studente` conosce il tipo `PianoStudi` non i suoi eventuali sottotipi
- `Studente` non creare l'istanza di `PianoStudi`



18

Prof. Tramontana - 26 Marzo 2025

Come vediamo in tale esempio quindi, viene creata una dipendenza(piano di studi) per studente, e gli viene iniettata nel costruttore, così che studente non conosce come essa è stata creata, ma avrà con se tale dipendenza.

STRUTTURALI->

ADAPTER(STRUTTURALE)->

Sappiamo che gli strutturali devono far compatire strutture dati tra loro, ma senza cambiare il codice, e questo significa che serve un adattatore(in questo caso l'adapter) che riesce a svolgere questo ruolo. Ci sono 4 principali attori in questo pattern tra cui:

-Target: L'interfaccia standard che il nostro programma si aspetta di utilizzare, e quindi nell'esempio della presa, sarebbe la forma della presa italiana(non quella nel muro, ma il caricatore);

-Client: è il pezzo di codice che cerca di svolgere un azione, usando le regole del target(caricare il telefono con la spina italiana);

-Adaptee: è la classe che serve a noi, che fa quello che serve a noi, ma parla una lingua diversa(presa del muro americana);

-Adapter: è la classe magica che creiamo, che si va ad incastrare con il client perché implementa il target, ma soprattutto traduce la richiesta, e chiama i metodi corretti nell'adaptee.

```

// 1. IL TARGET (La regola standard)
// Il nostro Computer accetta solo cose che rispettano questa interfaccia.
public interface PortaUSB {
    void inserisciUSB();
}

// IL CLIENT (Chi usa il codice)
// Il Computer è felice finché gli passi un oggetto di tipo PortaUSB.
public class Computer {
    public void leggiDati(PortaUSB dispositivo) {
        dispositivo.inserisciUSB();
        System.out.println("Computer: Dati letti con successo dalla porta USB!");
    }
}

// -----

// 2. L'ADAPTEE (L'intruso incompatibile)
// Questa è la nostra MicroSD. Ha metodi e comportamenti tutti suoi.
// Non rispetta la regola "PortaUSB".
public class MicroSD {
    public void inserisciNellaFessura() {
        System.out.println("MicroSD: Sono una schedina piccola, leggo i dati tramite il mio metodo");
    }
}

// 3. L'ADAPTER (Il nostro eroe)
// Si "traveste" da PortaUSB (implementa l'interfaccia)
// ma al suo interno nasconde e gestisce la MicroSD.
public class LettoreMicroSDAdapter implements PortaUSB {

    private MicroSD schedina;

    // Quando compriamo l'adattatore, ci infiliamo dentro la MicroSD
    public LettoreMicroSDAdapter(MicroSD schedina) {
        this.schedina = schedina;
    }

    // Questo è il metodo richiesto dal Computer.
    // L'adattatore lo "traduce" nel metodo della MicroSD!
    @Override
    public void inserisciUSB() {
        System.out.println("Adattatore: Sto convertendo i pin della MicroSD in un formato compatibile");
        schedina.inserisciNellaFessura(); // Chiama il metodo "straniero"
    }
}

```

FACADE(STRUTTURALI)->

Il facade è un pattern che ci facilita le cose, Il pattern Facade è un pattern strutturale che fornisce un'interfaccia unificata, semplificata e di alto livello per un intero sottosistema complesso di classi. Serve a nascondere la complessità interna di un sistema dietro un unico punto di accesso. In questo modo, il client interagisce solo con il Facade usando pochi metodi essenziali, senza dover conoscere, istanziare o far comunicare direttamente tra loro le singole classi del sottosistema, riducendo così l'accoppiamento. La parola chiave è che nasconde il caos dietro le quinte.

```

Java

public class TelecomandoUniversale {
    // Il Facade "nasconde" al suo interno tutti i pezzi complessi
    private Televisore tv;
    private ImpiantoAudio audio;
    private LettoreStreaming lettore;

    // Quando compri il telecomando, lo colleghi ai tuoi apparecchi
    public TelecomandoUniversale(Televisore tv, ImpiantoAudio audio, LettoreStreaming lettore) {
        this.tv = tv;
        this.audio = audio;
        this.lettore = lettore;
    }

    // IL BOTTONE MAGICO 1: Nasconde 6 operazioni complesse in un colpo solo!
    public void guardaFilm(String titolo) {
        System.out.println("\n--- Preparazione del salotto per il film ---");
        tv.accendi();
        tv.impostaSorgente("HDMI 1");
        audio.accendi();
        audio.impostaVolume(20);
        lettore.accendi();
        lettore.avviaFilm(titolo);
        System.out.println("--- Tutto pronto. Buona visione! ---\n");
    }

    // IL BOTTONE MAGICO 2: Spegne tutto con un click
    public void spegniTutto() {
        System.out.println("\n--- Spegnimento del cinema in casa ---");
        lettore.spegni();
        audio.spegni();
        tv.spegni();
        System.out.println("--- Buonanotte! ---\n");
    }
}

Java

public class Client {
    public static void main(String[] args) {

        // 1. Creiamo i nostri apparecchi (il caos)
        Televisore miaTv = new Televisore();
        ImpiantoAudio mioAudio = new ImpiantoAudio();
        LettoreStreaming mioLettore = new LettoreStreaming();

        // 2. Creiamo il Facade (il telecomando che maschera il caos)
        TelecomandoUniversale telecomando = new TelecomandoUniversale(miaTv, mioAudio, mioLettore);

        // 3. LA VITA DEL CLIENT ORA È BELLISSIMA:
        // Vuoi guardare un film? Chiami un solo metodo!
        telecomando.guardaFilm("Il Signore degli Anelli");

        // Hai finito? Chiami un solo metodo!
        telecomando.spegniTutto();
    }
}

```

BRIDGE(STRUTTURALI)->

Il bridge risolve un problema molto comune nella programmazione ad oggetti che è quella della moltiplicazione a dismisura delle sottoclassi. Il bridge quindi vorrebbe rendere indipendente un'astrazione(interfaccia, classe astratta...) dalla sua implementazione(sottoclassi), così che si possano fare le modifiche in modo indipendente. ES->

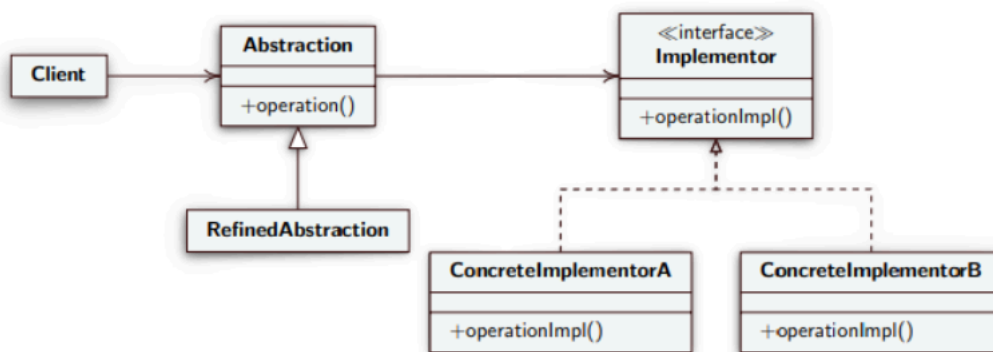
L'esempio pratico: Immagina di avere la classe base `Forma`. Da questa crei le classi figlie

Cerchio e Rettangolo . Fin qui tutto ok. Ora il capo ti dice: "Le forme devono avere un colore: Rosso o Blu".

- Senza il pattern Bridge, tu usi l'ereditarietà e crei: CerchioRosso , CerchioBlu , RettangoloRosso e RettangoloBlu . (Totale: 4 classi).
- Ora il capo vuole aggiungere il Triangolo . Devi creare TriangoloRosso e TriangoloBlu . (Totale: 6 classi).
- Ora il capo vuole aggiungere il colore Verde . Devi creare CerchioVerde , RettangoloVerde , TriangoloVerde . (Totale: 9 classi).
- Capisci il problema? **Il numero di classi si moltiplica a dismisura!** Se hai 10 forme e 10 colori, devi scrivere 100 classi diverse!

La soluzione sarebbe quella di dividere l'astrazione dall'implementazione, costruendo un ponte(bridge), dove la classe astratta forma contiene quadrato, rettangolo ecc.. e poi l'implementazione contiene le sue caratteristiche, così che si crea una forma, e poi si ha il ponte che lo fa puntare al colore.

SOLUZIONE->



Come vediamo qui, nel diagramma UML abbiamo diverse implementazioni tra cui:

-Abstraction: L'astrazione che è la nostra classe astratta "Forma" in questo caso, quindi è la classe generica di base di quello che è il tuo oggetto. Dentro questa classe c'è la funzione "Disegna" per fare la forma, ma al suo interno c'è anche il PONTE, si nota guardando la freccia che va da abstraction a implementor, e questo ponte dice che nella nostra classe c'è una variabile di tipo colore;

-RefinedAbstraction: Sono le classi figlie della nostra astrazione che semplicemente fanno la forma vera e propria;

-Implementor: L'interfaccia caratteristica che hai deciso di staccare, in modo tale che si possa creare il ponte, e al suo interno vi deve essere un metodo come "applicaTinta()".

-ConcreteImplementor A e B: Sono i colori veri e propri e fanno il lavoro pratico di colorare le forme.

```
public class Client {  
    public static void main(String[] args) {  
        Forma rettangoloBlu = new Rettangolo("Blu");  
        rettangoloBlu.disegna();  
        Forma cerchioRosso = new Cerchio("Rosso");  
        cerchioRosso.disegna();  
    }  
}
```

```
public abstract class Forma {  
    public abstract void disegna();  
    protected Disegno disegno;  
}
```

```
public class Rettangolo extends Forma {  
  
    public Rettangolo(String colore){  
        if(colore.equals("Rosso"))  
            this.disegno = new DisegnoRosso();  
        else if(colore.equals("Blu"))  
            this.disegno = new DisegnoBlu();  
    }  
  
    public void disegna() {  
        disegno.disegnaRettangolo();  
    }  
}
```

```

public class Cerchio extends Forma {

    public Cerchio(String colore){
        if(colore.equals("Rosso"))
            this.disegno = new DisegnoRosso();
        else if(colore.equals("Blu"))
            this.disegno = new DisegnoBlu();
        }

    public void disegna() {
        disegno.disegnaCerchio();
    }
}

```

```

public interface Disegno {
    void disegnaRettangolo();
    void disegnaCerchio();
}

```

```

public class DisegnoBlu implements Disegno {

    public void disegnaRettangolo() {
        System.out.println("Rettangolo blu.");
    }

    public void disegnaCerchio() {
        System.out.println("Cerchio blu.");
    }
}

```

```

public class DisegnoRosso implements Disegno {

    public void disegnaRettangolo() {
        System.out.println("Rettangolo rosso.");
    }

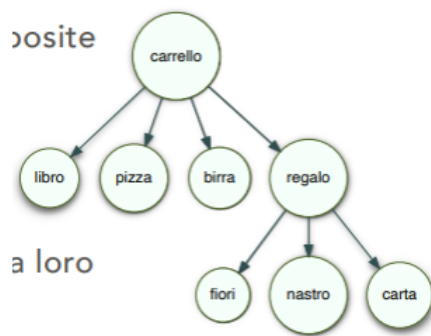
    public void disegnaCerchio() {
        System.out.println("Cerchio rosso.");
    }
}

```

COMPOSITE(STRUTTURALE)->

Il composite è un design pattern che ci permette di trattare uniformemente oggetti semplici e oggetti composti, significa che se c'è un oggetto composto che al suo interno contiene altri oggetti verrà trattato esattamente come un'oggetto singolo. Prendiamo l'esempio di una lista della spesa, che può variare da oggetti singoli come libro, birra ecc.. a oggetti complessi come il regalo che a sua volta ha all'interno la carta il nastro e un'oggetto. L'obiettivo è quello che invece che per sapere il prezzo del regalo si devono fare moltissimi controlli, si tratta quel regalo come un'oggetto semplice, entrambi avranno un metodo getprezzo(), solo che il

regalo a sua volta interrogherà con `getPrezzo()` quello che contiene così da trovare il prezzo totale.



Questo pattern possiede:

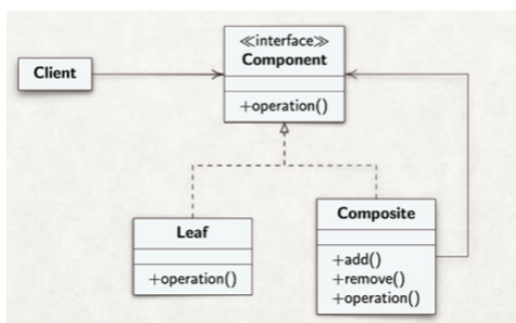
- Component: è l'interfaccia comune, che possiamo chiamare Articolo, e all'interno vi è il metodo `getPrezzo()` in comune per tutti;

- Leaf (La Foglia / L'Oggetto Semplice): è l'oggetto semplice(libro, birra) quindi che al suo interno non è come una matrioska quindi composite, ed è l'estensione dell'interfaccia(Component), e ha all'interno `getPrezzo()`;

- Composite (Il Contenitore / La Matrioska): è la nostra scatola regalo, che al suo interno ha altri elementi. Ovviamente estende anch'essa component, e ha `getPrezzo()`, ma guardando il diagramma UML, si vede come c'è una freccia che ritorna ad angolo retto su Component, perché questo Composite al suo interno ha tanti component, ed è per questo che si chiama ricorsivamente `getPrezzo()`. Avrà anche metodi come `add()` e `remove()`;

- Client: è il nostro carrello, a lui non interessa se è una foglia o un composite, a lui interessa solamente chiamare `getPrezzo()` sui prodotti.

PS-> I metodi `add()` e `remove()` si possono fare sia all'interno di component che fuori, ma questo comporta che se sono all'interno meno sicurezza perché si potrebbe avere `add()` sui leaf e non avrebbe senso, ma in compenso si ha più trasparenza perché il client non vede.



DECORATOR(STRUTTURALE)->

Questo design pattern detto anche wrapper, è un pattern che modifica la responsabilità di un oggetto a runtime, e rispetto all'adapter che mascherava un'oggetto, questo qui infila l'oggetto in una serie di scatole magiche, che lo fanno apparire come un'altro oggetto ma senza modificare il suo codice originale. L'esempio perfetto si può fare con una pizzeria:

I problema (Senza il Decorator)

Immagina di programmare il registratore di cassa di una pizzeria. Crei una classe base: `Margherita` (costa 5€).

Arriva un cliente e chiede una Margherita con **Salame**. Se usi la programmazione classica (l'ereditarietà), devi creare una nuova classe: `MargheritaConSalame` (costa 7€).

Arriva un altro e vuole **Salame e Funghi**. Devi creare un'altra classe: `MargheritaConSalameEFunghi` (costa 8€).

Capisci il problema? Se hai 20 ingredienti diversi, dovresti scrivere migliaia di classi per coprire tutte le combinazioni possibili (es. `MargheritaConSalameFunghiCipollaOlive...`). È un incubo!

La Soluzione magica (Il Decorator)

Il pattern Decorator dice: **smettiti di creare classi fisse per ogni combinazione!** Usa il trucco delle "Scatole Cinesi" (o matrioske).

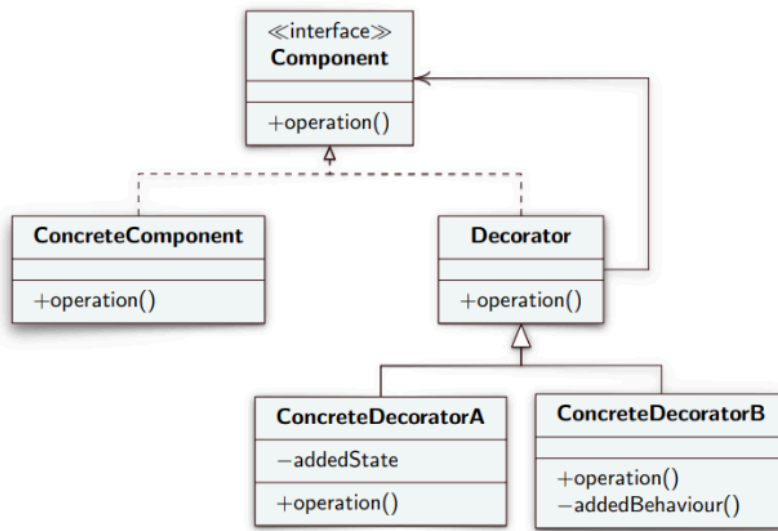
Funziona così:

1. Hai il tuo oggetto base: la **Pizza Margherita** (5€).
2. Gli ingredienti extra non sono classi intere di pizze, ma sono degli **"involucri"** (i Decorator). Hai l'involucro "Salame" (+2€) e l'involucro "Funghi" (+1€).

Quando il cliente vuole una Margherita con Salame e Funghi, tu non crei una nuova classe complicata, ma fai questo "pacchetto" al volo:

- Prendi la Margherita e la **infilati dentro l'involucro** Funghi.
- Prendi tutto questo e lo **infilati dentro l'involucro** Salame.

Quindi l'intento del decorator è prendere quel determinato oggetto, non modificarne il codice, oppure creare mille classi con tutte le sue combinazioni, ma semplicemente mettere questo oggetto dentro degli involucri, e quindi poterlo modificare a runtime.



In questo diagramma UML distinguiamo:

-Component: è la regola generale, quindi l'interfaccia cibo, che ci dice che tutti devono avere un metodo `getPrezzo()`;

-ConcreteComponent: è l'oggetto generale nudo, che noi vogliamo decorare, in questo caso la pizza margherita;

-Decorator: è la classe geniale che fa 2 cose contemporaneamente, dove estende il component, e al suo interno ne contiene un'altro di component, infatti vediamo la freccia che torna indietro, e quindi nasconde all'interno un altro cibo;

-ConcreteDecoratorA e B: Sono i veri e propri decoratori, quindi Funghi oppure salame. Quando il programma chiama il loro `getPrezzo`, prima calcolano il prezzo della pizza, e poi aggiungono il loro.

DIAGRAMMI UML->

Nei diagrammi UML dobbiamo prima capire chi entra in gioco per rappresentarli, e abbiamo:

Sostantivi: Rappresentano classi e attributi;

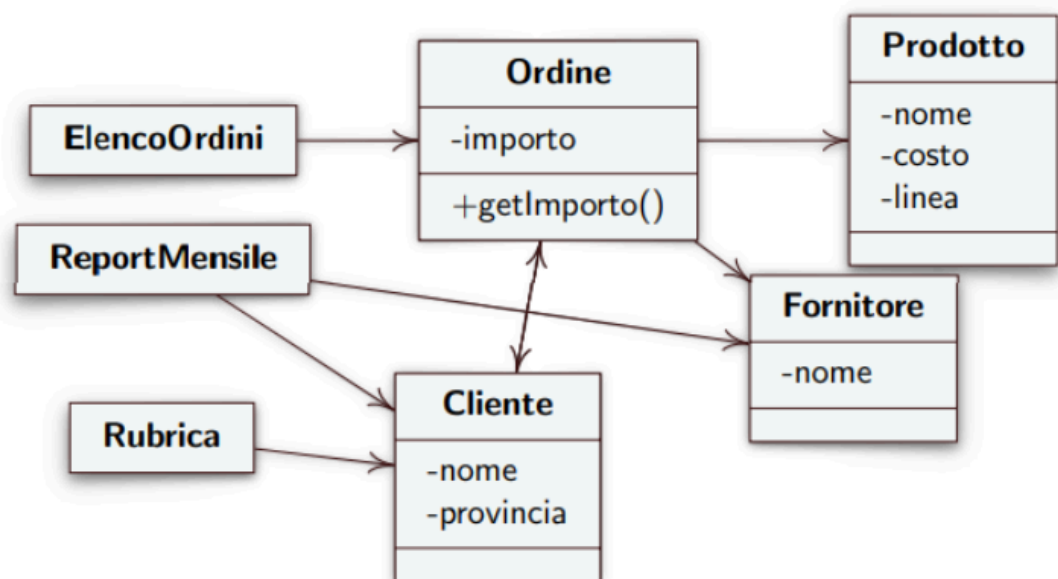
Verbi: Suggestiscono i metodi;

Per esempio, se abbiamo dei requisiti del tipo:

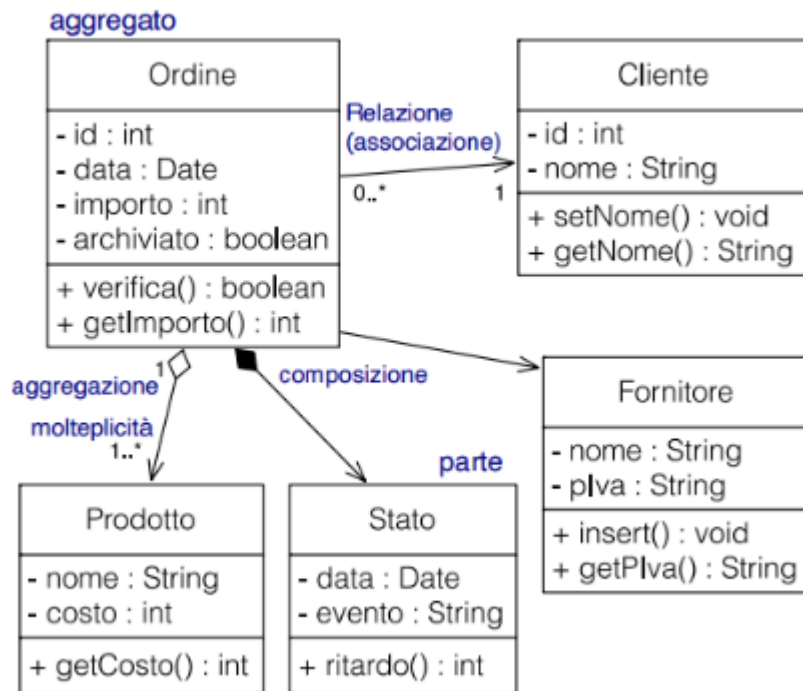
- ... dovrà essere possibile **cercare** un **cliente** e **mostrarne** i **dati anagrafici**
- ... la **scheda cliente mostra** i **dati** anagrafici ed **l'elenco** dei suoi **fornitori**
- ... su richiesta dell'utente si **calcola l'importo complessivo** degli **ordini** da lui **effettuati** in un intervallo di tempo
- ... ogni **ordine mostra nome fornitore, nome cliente, genere prodotti, importo complessivo**
- ... il **report mensile mostra** per ciascun **cliente provincia** di appartenenza e **totale** ordinato per ciascun **fornitore**

Individuiamo i seguenti ruoli:

- **Classi:** Cliente, Fornitore, Ordine, Prodotto, ReportMensile, SchedaCliente
- **Attributi:**
 - Cliente: dati anagrafici, nome, provincia
 - Prodotto: genere
 - Ordine: importo
 - Fornitore: nome
- **Metodi:**
 - Cercare un cliente
 - Mostrare dati anagrafici e fornitori per cliente
 - Calcolare totale ordini per un cliente
 - Selezionare ordini in un intervallo temporale
 - Calcolare totale ordini per un cliente per ciascun fornitore per mese



Qui vediamo come il diagramma UML è una rappresentazione del programma che noi stiamo svolgendo, e ci permette di vedere il collegamento tra le varie classi con annessi attributi all'interno, e poi i tipi di attributi che sono, perché ad esempio quando mettiamo (-) vuol dire che quell'attributo è privato. In questo diagramma non ci sono dettagli come in quello di sotto, dato che viene usato per far fare un brainstorming generale al cliente, ma non ci sono attributi che descrivono le associazioni tra classi, non ci sono i tipi negli attributi ecc..



La differenza tra questo diagramma sopra, e quello prima non è molto elevata, dato che descrivono più o meno la stessa cosa, solamente che questo diagramma sotto è un vero e proprio diagramma UML che serve ai programmatori per mostrargli in linea generale il programma, e di cosa si tratta. Qui rispetto a quello sopra troviamo i tipi negli attributi all'interno delle classi, e i metodi all'interno, e cosa restituiscono. I diagrammi UML sono simili agli schemi E-R di basi di dati, dato che hanno anche le cardinalità tra le classi. Ad esempio tra ordine e cliente vi sono 0... e 1, che significa che un cliente può avere da 0 a infiniti ordini, mentre un ordine può appartenere solo ad un cliente. Notiamo anche che al posto delle semplici frecce come sopra sono presenti i rombi in alcuni collegamenti, e tali rombi possono essere:

-Rombo vuoto-> Tra ordine e prodotto, significa che un ordine contiene dei prodotti, ma se cancello l'ordine i prodotti esisteranno lo stesso;

-Rombo pieno-> Tra ordine e stato, significa che quello stato appartiene solo a quell'ordine, se distruggo l'ordine distruggo anche lo stato.

DIAGRAMMA DI COLLABORAZIONE->

Mostra le interazioni tra gli oggetti, che comunicano scambiandosi messaggi che sono chiamate di funzione. La comunicazione avviene tramite frecce direzionate con etichetta, quindi con dei numerini che indicano l'ordine del flusso da seguire per capire il programma. In questo diagramma non vedi i tipi generali, ma gli oggetti vivi nel mentre che il programma sta eseguendo. Significa che non vedi solo la classe utente, ma Mario:utente. Per interpretare meglio tale diagramma vi è l'analisi grammaticale, che scompone le parole delle richieste e quindi crea classi o metodi in base agli attributi o verbi.

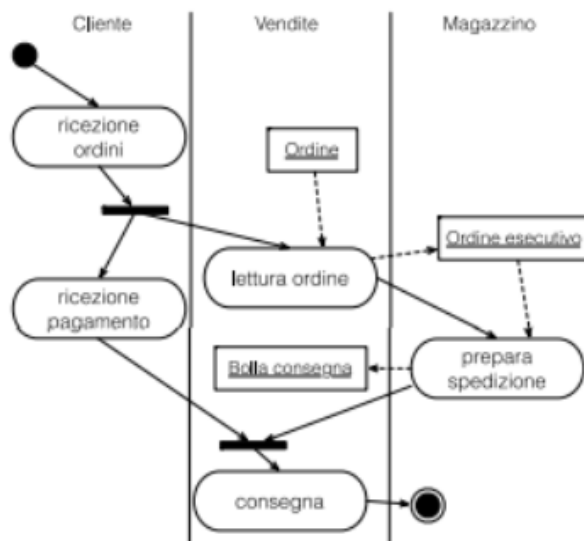
- I **Sostantivi** (nomi) diventano le Classi (es. "*Il Cliente fa un Ordine*" -> Classi Cliente e Ordine).

- I **Verbi** diventano i Metodi (es. "*Il sistema **calcola** il totale*" -> metodo `calcolaTotale()`).

Se però ti limiti alle parole del cliente il programma sarà fragile, ed è per questo che il progettista deve aggiungere delle classi invisibili(design pattern) per rendere l'architettura robusta.

DIAGRAMMA DELLE ATTIVITA'->

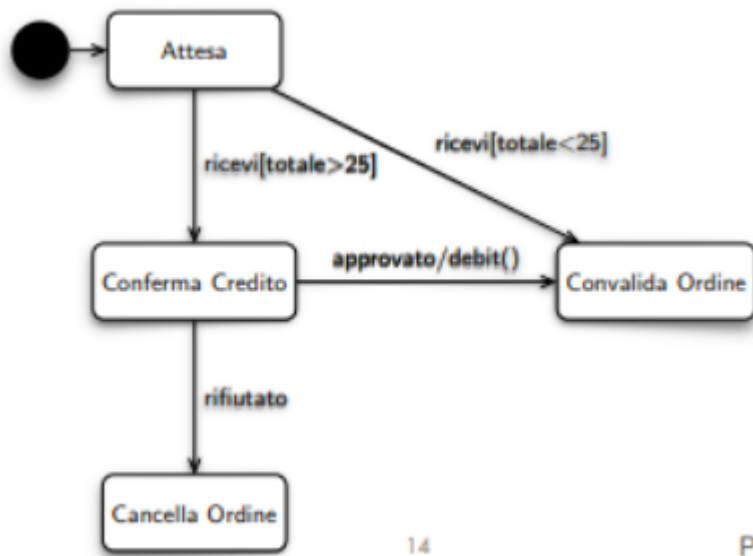
Il diagramma delle attività è il manuale delle istruzioni del nostro programma, ed è un diagramma di flusso potenziato.



Guardando l'immagine notiamo come tale diagramma viene usato per delineare le attività del programma software, ma non ci dice quali oggetti svolgono tali attività, ma può essere una base per costruire il diagramma di collaborazione tra oggetti. Le tre corsie principali che vediamo in foto ci permettono di capire chi fa cosa. Vediamo che sono collegati da una freccia, ma ognuno sta nella propria corsia. Il pallino nero indica che il processo inizia da lì. I rettangoli arrotondati sono le azioni vere e proprie. Il cerchio nero con il bordo attorno invece è dove finisce l'esecuzione del processo. La barra nera che troviamo dopo ricezione ordini e prima di consegna, sdoppia le azioni, e indica che quelle due azioni stanno avvenendo contemporaneamente, quindi mentre dopo l'ordine il cliente fa il pagamento, il reparto vendite legge l'ordine. L'altra barra nera invece ricongiunge le due attività che si erano sdoppiate, quindi fino a che non finiscono entrambe l'attività non potrà finire. I rettangoli normali non arrotondati sono oggetti o dati. Le frecce tratteggiate indicano chi sta leggendo o stampando da quel foglio.

DIAGRAMMA DEGLI STATI->

Il diagramma degli stati prende un singolo oggetto e ti fa vedere tutte le sue fasi di vita fino al compimento della sua attività.

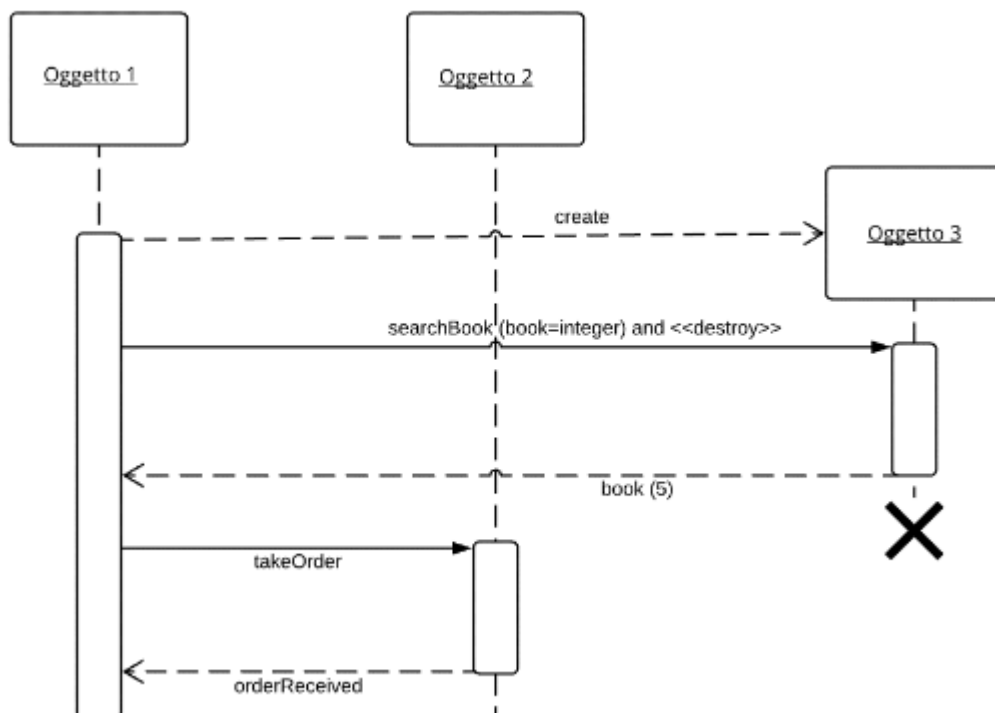


Tali rettangoli arrotondati indicano in che stato si trova l'oggetto, e il pallino nero al solito indica l'avvio della sua esecuzione. Le frecce indicano lo stato per la quale si passa da uno stato un altro, e indicano evento[Condizione] azione. Uno stato a volte può essere talmente complesso da avere sottostati al suo interno. Lo stato può essere:

-Stato composto sequenziale: Dove questi stati annidati avvengono uno dopo l'altro, come ad esempio gli stati di una torta, dove prima abbiamo lievitazione e poi doratura;

-Stato composto concorrente: Questo stato invece ha 2 sottostati che avvengono contemporaneamente, quindi ad esempio uno studente in sessione può studiare analisi ed entrare in panico nello stesso momento.

DIAGRAMMA UML DI SEQUENZA->



Il diagramma UML di sequenza è un diagramma che descrive come gli oggetti comunicano passandosi messaggi, ma lungo il tempo, quindi il tempo è un'importante variabile. In orizzontale troviamo gli oggetti che partecipano al programma, invece in verticale

troviamo il tempo che passa inesorabile, ciò significa che più una freccia è in basso e più accade tardi nel tempo, e quindi non c'è più bisogno di numerare queste frecce. La linea che pende giù da ogni oggetto è la linea della vita che indica che l'oggetto esiste. Quando tale linea tratteggiata non è collegata a niente significa che l'oggetto esiste ma non sta facendo niente, invece se quella linea è collegata alla barra di attivazione(rettangolo verticale) significa che sta eseguendo un'azione, e appena finisce torna a dormire. Ci sono vari tipi di frecce:

-Freccia piena-> è una chiamata a un metodo;

-Freccia tratteggiata-> è un valore di ritorno, significa ho finito questo, ti ritorno il valore.